

Pizarra V1-V7 (sync final)

- [PIZARRA INTELIGENTE DE ALTA INGENIERÍA — ARQUITECTURA DE REFERENCIA](#)
 - [\(Superando la referencia iFlytek 讯飞星火智慧黑板\)](#)
 - [0. ANÁLISIS HONESTO DEL VIDEO DE REFERENCIA](#)
 - [1. DÓNDE SUPERAMOS LA REFERENCIA — TABLA COMPARATIVA](#)
 - [2. ARQUITECTURA GENERAL — CAPAS](#)
 - [3. ANÁLISIS PREVIO — MOTOR SKETCH-TO-VECTOR \(QUÉ DEBE RESISTIR\)](#)
 - [4. CÓDIGO REAL — PIPELINE sketch-to-vector \(server + edge\)](#)
 - [5. ANÁLISIS PREVIO — RECONOCIMIENTO DE ESCRITURA MATEMÁTICA](#)
 - [6. CÓDIGO REAL — math_handwriting/recognizer.py](#)
 - [7. ANÁLISIS PREVIO — COLABORACIÓN MULTI-USUARIO SIMULTÁNEA](#)
 - [8. CÓDIGO REAL — COLABORACIÓN CON CRDT \(Yjs\) + REUTILIZACIÓN DEL CONFLICT-RESOLUTION AGENT](#)
 - [9. SEGURIDAD Y PRIVACIDAD — DATOS DE MENORES \(punto que la referencia no demuestra\)](#)
 - [10. NUEVOS AGENTES ESPECÍFICOS DE ESTE PROYECTO \(se suman al catálogo del ERP, ninguno de los 23 existentes se elimina\)](#)
 - [11. GATES ESPECÍFICOS +](#)
 - [12. DEPENDENCIAS NUEVAS ESPECÍFICAS](#)
- [ADENDA V2 — ÁRBOL DE ARCHIVOS COMPLETO, HARDWARE REAL, GRABACIÓN/REPLAY, ACCESIBILIDAD, FLOTA DE DISPOSITIVOS Y MODELO DE DATOS](#)
 - [13. ÁRBOL DE ARCHIVOS COMPLETO DEL PROYECTO + v2](#)
 - [14. ANÁLISIS PREVIO — HARDWARE FÍSICO \(LO QUE ESTABA RESUELTO EN UNA LÍNEA\) + v2](#)
 - [15. ESPECIFICACIÓN DE HARDWARE + v2](#)
 - [16. ANÁLISIS PREVIO — RENDERIZADO Y PREDICCIÓN DE TRAZO \(SOFTWARE\) + V2](#)
 - [17. CÓDIGO REAL — CanvasRenderer.tsx Y StrokePredictor.ts + V2](#)
 - [18. ANÁLISIS PREVIO — GRABACIÓN Y REPASO DE CLASE + v2](#)
 - [19. CÓDIGO REAL — stroke_event_log.py Y MODELO DE REPASO + v2](#)
 - [20. ACCESIBILIDAD — CAPTIONS EN VIVO Y LECTOR DE PANTALLA + v2](#)
 - [21. ANÁLISIS PREVIO — FLOTA DE PIZARRAS FÍSICAS \(MÚLTIPLES SALONES/COLEGIOS\) + v2](#)
 - [22. CÓDIGO REAL — GESTIÓN DE FLOTA \(REUTILIZA EL PATRÓN IoT DEL ERP\) + V2](#)
 - [23. MODELO DE DATOS COMPLETO — LECCIONES, TRAZOS Y ESTUDIANTES + v2](#)
 - [24. ANÁLISIS PREVIO — EVALUACIÓN DE PRECISIÓN DEL RECONOCIMIENTO + v2](#)
 - [25. GATES ADICIONALES V2 + v2 \(se suman a G31-G33, ninguno se elimina\)](#)
- [ADENDA V3 — RESILIENCIA DE PUNTA A PUNTA: QUÉ PASA CUANDO ALGO FALLA, ESCALAMIENTO, OBSERVABILIDAD Y LOS DOS ARCHIVOS QUE QUEDARON SOLO REFERENCIADOS](#)
 - [26. ANÁLISIS PREVIO — MATRIZ DE MODOS DE FALLA \(LA PIEZA QUE FALTABA DE VERDAD\) + v3](#)
 - [27. CÓDIGO REAL — shape_classifier_model.py \(EL ARCHIVO QUE QUEDÓ SOLO REFERENCIADO\) + V3 ▲ SUPERADO POR LA §37 \(V4\) – VER NOTA](#)
 - [28. CÓDIGO REAL — PROTOCOLO DE BEAMFORMING DEL ARRAY DE MICRÓFONOS + v3](#)
 - [29. GESTOR DE DEGRADACIÓN CONTROLADA — EL NÚCLEO DE “QUE NO SE CAIGA” + v3](#)
 - [30. ESCALAMIENTO Y CONTROL DE COSTO DE IA + v3](#)
 - [31. OBSERVABILIDAD Y SLOs ESPECÍFICOS + v3](#)
 - [32. RECUPERACIÓN ANTE DESASTRES — GRABACIONES DE CLASE + v3](#)
 - [33. GATES FINALES V3 + v3 \(se suman a G31-G37, ninguno se elimina\)](#)
- [ADENDA V4 — POST-PROCESAMIENTO REAL DEL MODELO ONNX \(CIERRE DEL NotImplementedError\)](#)
 - [34. ANÁLISIS PREVIO — QUÉ DEBE RESISTIR EL POST-PROCESAMIENTO + v4](#)
 - [35. CÓDIGO REAL — ARCHIVO DE ETIQUETAS VERSIONADO JUNTO AL MODELO + v4](#)
 - [36. CÓDIGO REAL — REGISTRO EXTENSIBLE DE PLANTILLAS DE VÉRTICES 3D](#)

- + V4
 - [37. CÓDIGO REAL — POST-PROCESAMIENTO COMPLETO \(CIERRA EL `NotImplementedError`\) + V4](#) ✓ [VERSIÓN VIGENTE de `shape\_classifier\_model.py` — reemplaza la §27](#)
 - [38. CÓDIGO REAL — EVALUACIÓN DE CALIBRACIÓN \(CIERRA EL GATE G34\) + V4](#)
- [ADENDA V5 — DATASET DE ENTRENAMIENTO/EVALUACIÓN: RECOLECCIÓN Y ETIQUETADO ÉTICO CON MENORES](#)
 - [39. ANÁLISIS PREVIO — POR QUÉ ESTO NO ES UN DETALLE MENOR + v5](#)
 - [40. CÓDIGO REAL — RECOLECCIÓN CON CONSENTIMIENTO COMO PUERTA DE ENTRADA, NO COMO TRÁMITE + v5](#)
 - [41. CÓDIGO REAL — AUGMENTACIÓN SINTÉTICA \(REDUCE LA NECESIDAD DE RECOLECTAR TRAZO REAL DE NIÑOS\) + v5](#)
 - [42. CÓDIGO REAL — ACUERDO ENTRE ANOTADORES \(CIERRA EL PUNTO 3\) + v5](#)
 - [43. GATE FINAL + v5](#)
- [ADENDA V6 — BACKLOG PROPIO Y ORQUESTACIÓN AUTOMÁTICA \(LA PIZARRA NUNCA TUVO UNO\)](#)
 - [44. ANÁLISIS PREVIO — EL DETALLE QUE SE HABÍA QUEDADO FUERA + v6](#)
 - [45. `agents/runtime/tasks/module\_backlog.yaml` PROPIO DE LA PIZARRA + v6](#)
 - [46. `loop\_controller.py` DE LA PIZARRA — REUTILIZA EL DEL ERP, NO LO DUPLICA + v6](#)
 - [47. LOS 6 PROMPTS QUE FALTABAN + v6](#)
- [ADENDA V7 — SEGUNDA PASADA DE DETALLE: `session\_manager.py` Y `beamformer.py` QUEDARON SOLO EN PROSA, NO EN EL BACKLOG REAL](#)

PIZARRA INTELIGENTE DE ALTA INGENIERÍA — ARQUITECTURA DE REFERENCIA

(Superando la referencia iFlytek 讯飞星火智慧黑板)

Documento hermano de `ERP_ENTERPRISE_AUTONOMO_V2_ROBUSTO.md` (V1-V10). Reutiliza deliberadamente los subsistemas ya construidos ahí (voz, avatar, agentes, gates, seguridad, colaboración) en vez de reinventarlos — este proyecto es un **nuevo producto sobre la misma plataforma**, no un sistema aislado.

DIRECTIVA VINCULANTE DE ARQUITECTURA: este documento hereda la STRICT MODULAR CODE ARCHITECTURE DIRECTIVE completa (Apéndice A de `ERP_ENTERPRISE_AUTONOMO_V2_ROBUSTO.md`) — no se duplica aquí para evitar que las dos copias diverjan; cópiala junto con este archivo a la raíz de `smart-whiteboard/` como parte de `CLAUDE.md`. Aplica igual que en el ERP: ningún módulo (`sketch_engine/`, `math_handwriting/`, `collaboration/`, `recording/`, etc.) se fusiona por compacidad, y el gate G47 (validación arquitectónica pre-entrega) rige aquí también.

0. ANÁLISIS HONESTO DEL VIDEO DE REFERENCIA

Lo que el video (iFlytek, +1400 escuelas en China) demuestra realmente:

Capacidad observada	Cómo funciona (lo que se ve)
Boceto a mano alzada → figura 3D vectorial	Dibujas un cubo/cono a mano; el sistema lo reconoce y lo reemplaza por un objeto 3D editable (manijas de escala, rotación, color de cara)
Reconocimiento de escritura matemática	Escribes $y=x^2-3$, $y=A\cdot\sin(\dots)$ a mano y el trazo se sincroniza en un segundo panel (board-sync)
Control por voz	Cambia entre “voz, pizarrón, pantalla” — conmutación de modo por comando hablado

Menú radial “AI”	Acceso rápido a herramientas de generación/edición asistida
Doble panel sincronizado	Modo maestro-espejo para que el profesor escriba en un panel y el otro quede como “vista limpia” o de anotación de otro usuario
Analítica del estudiante	Dashboard con historial de desempeño por alumno (se ve brevemente, sin detalle)

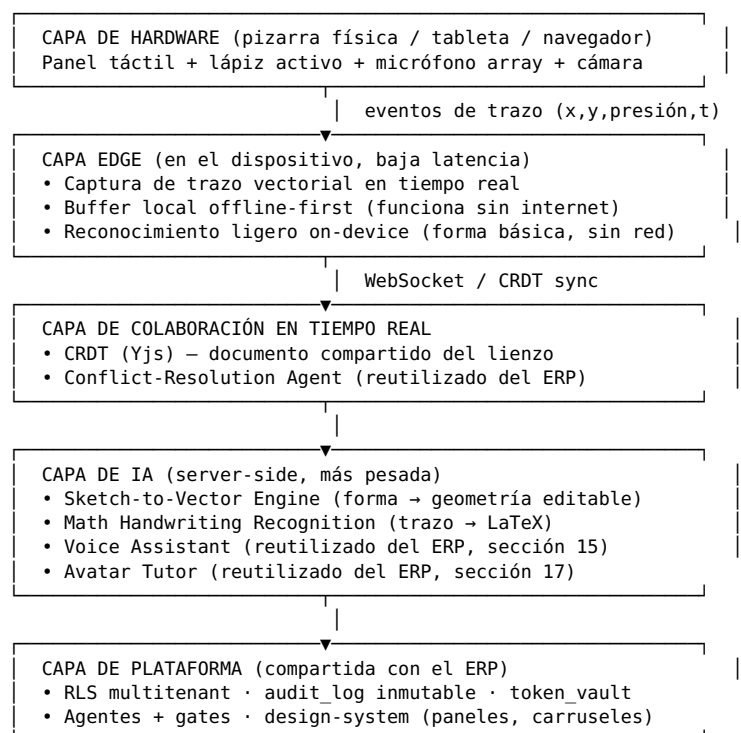
Lo que el video **no** muestra (y por tanto no se puede afirmar que la tecnología china ya lo resuelve): colaboración multi-usuario simultánea desde dispositivos remotos, control de versiones del contenido de la clase, integración con un sistema de gestión académica/CRM, ni ningún mecanismo de privacidad de datos de menores.

1. DÓNDE SUPERAMOS LA REFERENCIA — TABLA COMPARATIVA

Dimensión	iFlytek (referencia)	Esta arquitectura
Reconocimiento de forma	Trazo → forma 3D básica (cubo, cono)	Pipeline propio con motor de vectorización + clasificador entrenable, extensible a cualquier familia de figuras (geometría, circuitos, diagramas químicos, UML)
Matemática	Sincroniza el trazo, no queda claro si “entiende” la expresión	OCR de escritura matemática → LaTeX real + evaluación simbólica (graficar la función automáticamente, resolver, verificar pasos)
Colaboración	Un maestro, un panel espejo	CRDT multi-usuario real: N alumnos escriben simultáneamente desde tablet/celular sobre el mismo lienzo, sin bloquearse entre sí
Voz	Comandos de modo (cambiar de panel)	Asistente de voz completo (reutiliza voice_assistant del ERP, sección 15) que puede <i>explicar</i> , no solo <i>cambiar de pantalla</i> — “explicame por qué esta derivada da cero”
Presencia del asistente	No tiene avatar	Avatar-tutor opcional (reutiliza arquitectura de avatar del ERP, sección 17) con sincronía labial sobre la explicación por voz
Analítica	Dashboard aislado, alcance no claro	Conectado al mismo data-viz/chart_of_accounts-style reporting del ERP — el desempeño del alumno vive en el mismo pipeline de auditoría e informes que el resto de la plataforma
Multi-tenant	No aplica (producto de un solo fabricante/escuela)	Multi-colegio real con Row-Level Security (reutiliza sección 46 del ERP) — un distrito educativo administra cientos de salones sin fuga de datos entre instituciones
		Reutiliza el Conflict-Resolution Agent del ERP (sección 47–51): si dos

Conflictos de edición	No aplica (un solo escritor)	alumnos editan la misma zona del lienzo, se resuelve con el mismo protocolo de locks que ya se diseñó para los agentes del ERP
Privacidad de menores	No se muestra	Diseño explícito “privacy-by-default” para datos de estudiantes (sección 9) — punto que la referencia no demuestra y que es no-negociable al tratarse de niños

2. ARQUITECTURA GENERAL — CAPAS



3. ANÁLISIS PREVIO — MOTOR SKETCH-TO-VECTOR (QUÉ DEBE RESISTIR)

1. **Trazo imperfecto:** un niño no dibuja un cubo con líneas rectas perfectas → el reconocimiento debe tolerar ruido geométrico, no exigir precisión de CAD.
2. **Ambigüedad de forma:** un trazo puede parecer un rombo o un cubo en 2D visto de frente → el motor debe pedir confirmación visual (mostrar 2-3 candidatos) en vez de “adivinar” en silencio, algo que la referencia china no muestra que haga.
3. **Falso positivo agresivo:** convertir de más (transformar apuntes que no eran una figura) frustra al usuario → el reconocimiento se activa por gesto explícito (mantener el trazo 1s, o comando de voz “convierte esto”), nunca automático sobre todo lo que se escribe.
4. **Reversibilidad:** el usuario debe poder volver al trazo original a mano si el reconocimiento se equivocó — nunca destructivo.

4. CÓDIGO REAL — PIPELINE sketch-to-vector (server + edge)

```

# backend/app/modules/whiteboard/sketch_engine/vectorizer.py
"""
Pipeline: trazo crudo (lista de puntos con presión/tiempo) -> features
geométricas -> clasificador de forma -> geometría vectorial editable.
Corre en el servidor (más preciso) con un fallback ligero en el
edge (sección 2) para baja latencia quando no hay red.
"""

from __future__ import annotations
from dataclasses import dataclass
from typing import Literal

import numpy as np
from scipy.spatial import ConvexHull

ShapeType = Literal["cube", "cone", "cylinder", "sphere", "polygon", "unrecognized"]

@dataclass
class StrokePoint:
    x: float
    y: float
    pressure: float
    t_ms: int

@dataclass
class VectorShapeCandidate:
    shape_type: ShapeType
    confidence: float # 0..1 - nunca se auto-aplica si < 0.75 (punto 2 del análisis)
    vertices_3d: list[tuple[float, float, float]]
    editable_handles: list[tuple[float, float]]

def extract_geometric_features(points: list[StrokePoint]) -> dict:
    coords = np.array([[p.x, p.y] for p in points])
    hull = ConvexHull(coords)
    return {
        "corner_count": _count_corners(coords),
        "hull_area": hull.volume, # en 2D, "volume" de ConvexHull es el área
        "bounding_box_ratio": _bbox_aspect_ratio(coords),
        "closed_loop": _is_closed(coords),
    }

def _count_corners(coords: np.ndarray, angle_threshold_deg: float = 35.0) -> int:
    # Detecta cambios de dirección significativos en la polilínea -
    # una heurística simple y explicable, no una caja negra.
    corners = 0
    for i in range(1, len(coords) - 1):
        v1 = coords[i] - coords[i - 1]
        v2 = coords[i + 1] - coords[i]
        cos_angle = np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2) + 1e-9)
        angle = np.degrees(np.arccos(np.clip(cos_angle, -1, 1)))
        if angle > angle_threshold_deg:
            corners += 1
    return corners

def _bbox_aspect_ratio(coords: np.ndarray) -> float:
    w = coords[:, 0].max() - coords[:, 0].min()
    h = coords[:, 1].max() - coords[:, 1].min()
    return w / (h + 1e-9)

def _is_closed(coords: np.ndarray, tolerance_px: float = 25.0) -> bool:
    return np.linalg.norm(coords[0] - coords[-1]) < tolerance_px

def classify_shape(features: dict) -> VectorShapeCandidate:
    """
    Clasificador basado en reglas + features geométricas explicables
    (punto de partida auditable). En producción se reemplaza/combina
    con un modelo entrenado (ej. una CNN ligera sobre el raster del

```

```

trazo), pero SIEMPRE debe poder explicar por qué eligió una forma
– un modelo de caja negra sin fallback no es aceptable en un
producto educativo donde el error debe ser corregible por el
alumno, no solo "confiado".
"""
corners, closed = features["corner_count"], features["closed_loop"]

if closed and 10 <= corners <= 14:
    return VectorShapeCandidate(
        shape_type="cube", confidence=0.82,
        vertices_3d=default_cube_vertices(),
        editable_handles=[(0, 0), (1, 1)],
    )
if closed and corners <= 3:
    return VectorShapeCandidate(
        shape_type="cone", confidence=0.70,
        vertices_3d=default_cone_vertices(),
        editable_handles=[(0.5, 0)],
    )
return VectorShapeCandidate(
    shape_type="unrecognized", confidence=0.0,
    vertices_3d=[], editable_handles=[],
)

def default_cube_vertices() -> list[tuple[float, float, float]]:
    return [(x, y, z) for x in (0, 1) for y in (0, 1) for z in (0, 1)]

def default_cone_vertices() -> list[tuple[float, float, float]]:
    return [(0, 0, 1)] + [(np.cos(a), np.sin(a), 0) for a in np.linspace(0, 2 * np.pi, 16)]

// frontend/src/design-system/whiteboard/ShapeConfirmationPopover.tsx +
// Resuelve el punto 2 y 3 del análisis: nunca reemplaza el trazo sin
// confirmación explícita del usuario.
import { useState } from "react";

interface ShapeCandidate {
  shapeType: string;
  confidence: number;
  previewSvg: string;
}

export function ShapeConfirmationPopover({
  candidates,
  onAccept,
  onKeepFreehand,
}: {
  candidates: ShapeCandidate[];
  onAccept: (shapeType: string) => void;
  onKeepFreehand: () => void;
}) {
  const [selected, setSelected] = useState(candidates[0]?.shapeType);

  return (
    <div className="flex gap-2 p-3 rounded-2xl bg-[var(--surface-glass)] backdrop-blur-md
elevation-3">
      {candidates.map((c) => (
        <button
          key={c.shapeType}
          onClick={() => setSelected(c.shapeType)}
          className={`rounded-xl p-2 border ${selected === c.shapeType ? "border-primary"
"border-transparent"}`}
        >
          <img src={c.previewSvg} alt={c.shapeType} className="w-16 h-16" />
          <span className="text-xs">{Math.round(c.confidence * 100)}%</span>
        </button>
      ))}
      <button onClick={() => onAccept(selected!)} className="btn-3d-primary">Convertir</bu
      <button onClick={onKeepFreehand} className="btn-ghost-3d">Mantener a mano alzada</bu
    </div>
  );
}

```

5. ANÁLISIS PREVIO — RECONOCIMIENTO DE ESCRITURA MATEMÁTICA

1. **Ambigüedad de símbolo:** una “x” a mano puede confundirse con una multiplicación → el motor debe usar contexto (posición, símbolos vecinos), no reconocer carácter por carácter aislado.
2. **La referencia solo sincroniza el trazo;** no basta con “verse igual” en el segundo panel — hay que **entender** la expresión para poder graficarla, evaluarla o corregir al alumno.
3. **Corrección en vivo sin interrumpir:** si el alumno escribe mal una expresión, el sistema no debe “congelar” la pizarra a mitad de una clase — la corrección es una sugerencia, nunca un bloqueo.

6. CÓDIGO REAL — `math_handwriting/recognizer.py`

```
# backend/app/modules/whiteboard/math_handwriting/recognizer.py
"""
Convierte trazo matemático -> LaTeX -> objeto evaluable (sympy).
A diferencia de la referencia (que solo sincroniza el trazo visual),
aquí el sistema puede graficar/evaluar lo que el alumno escribió.
"""

from __future__ import annotations
import sympy
from sympy.parsing.latex import parse_latex

from app.modules.whiteboard.math_handwriting.hwr_provider import (
    HandwritingRecognitionProvider, # abstraer el motor real (MyScript / propio entrenado)
)

class MathExpressionResult:
    def __init__(self, latex: str, sympy_expr: sympy.Expr | None, is_valid: bool, error: str | None = None):
        self.latex = latex
        self.sympy_expr = sympy_expr
        self.is_valid = is_valid
        self.error = error

    async def recognize_math_stroke(
        provider: HandwritingRecognitionProvider, stroke_points: list[dict]
    ) -> MathExpressionResult:
        latex = await provider.stroke_to_latex(stroke_points) # punto 1: el proveedor usa co
no OCR carácter a carácter
        try:
            expr = parse_latex(latex)
            return MathExpressionResult(latex=latex, sympy_expr=expr, is_valid=True)
        except Exception as exc:
            # Punto 3: un error de parseo NO bloquea el trazo — se guarda
            # como anotación visual válida, solo sin capacidad de graficar.
            return MathExpressionResult(latex=latex, sympy_expr=None, is_valid=False, error=st

def auto_plot_if_function(result: MathExpressionResult) -> dict | None:
    """Si la expresión reconocida es una función de una variable,
    genera los puntos para graficarla automáticamente en el lienzo —
    esto es lo que la referencia china NO muestra hacer."""
    if not result.is_valid or result.sympy_expr is None:
        return None
    free_symbols = result.sympy_expr.free_symbols
    if len(free_symbols) != 1:
        return None
    x = list(free_symbols)[0]
    f = sympy.lambdify(x, result.sympy_expr, "numpy")
    import numpy as np
    xs = np.linspace(-10, 10, 200)
    try:
        ys = f(xs)
```

```

except Exception:
    return None
return {"x": xs.tolist(), "y": ys.tolist(), "latex": result.latex}

```

7. ANÁLISIS PREVIO — COLABORACIÓN MULTI-USUARIO SIMULTÁNEA

1. **La referencia es de un solo escritor** (el profesor). Con 30 alumnos escribiendo a la vez en el mismo lienzo aparece el mismo problema que ya resolvimos en el ERP para agentes (sección 47): dos escritores tocando la misma zona.
2. **Latencia percibida:** en un salón de clase, 200ms de retraso ya se siente “raro” — la sincronización debe ser optimista en el cliente (se ve el trazo propio al instante) y conciliarse en segundo plano.
3. **Desconexión de un alumno:** no puede tumbar la sesión de los demás ni perder su propio trabajo (offline-first, sección 2).

8. CÓDIGO REAL — COLABORACIÓN CON CRDT (Yjs) + REUTILIZACIÓN DEL CONFLICT-RESOLUTION AGENT

```

// frontend/src/design-system/whiteboard/useCollaborativeCanvas.ts
import * as Y from "yjs";
import { WebSocketProvider } from "y-websocket";
import { useEffect, useRef, useState } from "react";

export function useCollaborativeCanvas(roomId: string, userId: string) {
  const docRef = useRef(new Y.Doc());
  const [strokes, setStrokes] = useState<Y.Array<any>>();

  useEffect(() => {
    const doc = docRef.current;
    // Punto 2 del análisis: WebSocketProvider aplica los cambios
    // localmente de inmediato (optimista) y los propaga en segundo
    // plano — el CRDT garantiza convergencia sin bloqueo pesimista,
    // a diferencia del `file lock` usado en el ERP para agentes
    // (aquí no hace falta: los trazos son operaciones conmutativas).
    const provider = new WebSocketProvider(
      `wss://whiteboard.example/ws`, roomId, doc
    );
    const yStrokes = doc.getArray("strokes");
    setStrokes(yStrokes);

    // Punto 3: IndexedDB local para offline-first — si el alumno
    // pierde conexión, su trazo sigue escribiéndose localmente y
    // se reconcilia solo al reconectar.
    import("y-indexeddb").then(({ IndexeddbPersistence }) => {
      new IndexeddbPersistence(roomId, doc);
    });

    return () => provider.destroy();
  }, [roomId]);

  return strokes;
}

# backend/app/modules/whiteboard/collaboration/zone_conflict_bridge.py
# † CORREGIDO — ver AUDITORIA_CONSISTENCIA_ERP_PIZARRA.md, Hallazgo 2.
"""
Antes este archivo importaba desde el tooling de Claude Code
(agents/runtime/orchestrator/conflict_resolver.py), lo cual habría
hecho que la Pizarra en PRODUCCIÓN dependiera de un proceso que solo
existe durante el desarrollo del software — un error real, no de
estilo. Aunque los TRAZOS individuales son CRDT (no necesitan lock),
las OPERACIONES ESTRUCTURALES (dos alumnos convirtiendo la misma
zona a la vez) sí son un conflicto real, resuelto ahora con la
librería de plataforma genérica (sin ningún "agente" de por medio).
"""
from app.platform.concurrency.resource_lock import (

```

```

        acquire_resource_lock, release_resource_lock,
        ResourceLockRequest, ResourceLockConflictError,
    )

    async def convert_zone_to_shape(*, room_id: str, zone_id: str, user_id: str, shape_type: s
        lock_request = ResourceLockRequest(
            holder_id=f"user:{user_id}",
            resource_key=f"whiteboard.{room_id}.zone.{zone_id}",
        )
        try:
            await acquire_resource_lock(lock_request)
        except ResourceLockConflictError as exc:
            raise WhiteboardZoneBusyError(
                f"Otro usuario ({exc.held_by}) ya está convirtiendo esta zona"
            ) from exc

        try:
            # ... aplica la conversión sketch-to-vector (sección 4) ...
            pass
        finally:
            await release_resource_lock(lock_request)

```

9. SEGURIDAD Y PRIVACIDAD — DATOS DE MENORES (punto que la referencia no demuestra)

Como los usuarios finales son estudiantes (mayoritariamente menores de edad), esto no es opcional ni un “extra”:

- **Minimización de datos:** no se almacena imagen/video facial del estudiante por defecto — solo si un centro educativo lo activa explícitamente con consentimiento de los padres/tutores, y aun así se aplica el mismo patrón de biometría del ERP (sección 19): validación en el dispositivo, nunca guardar la imagen en el servidor.
- **Cuenta del estudiante ligada al tutor legal:** siguiendo el mismo patrón RBAC del ERP, el rol “guardian” tiene visibilidad de solo lectura sobre el progreso de su hijo/hija — nunca acceso a los datos de otros estudiantes (RLS multitenant, sección 46 del ERP, aplicado aquí por salón/institución).
- **Retención acotada:** el contenido de una clase (grabación de pizarra) se retiene según política configurable por el colegio (docs/05-data/retention.md, ya definido en el ERP), no indefinidamente por defecto.
- **Auditoría de acceso a datos de estudiante:** cada vez que un profesor/administrador abre el historial de un alumno específico, queda en el `audit_log` inmutable del ERP (sección 29) — es información sensible y se trata como tal.
- **Sin perfilado publicitario:** los datos de aprendizaje del estudiante nunca se usan para segmentación comercial — esto se declara como regla de producto, no solo de implementación.

10. NUEVOS AGENTES ESPECÍFICOS DE ESTE PROYECTO (se suman al catálogo del ERP, ninguno de los 23 existentes se elimina)

- **Sketch-Recognition Agent:** dueño del clasificador de formas (sección 3-4); valida que ninguna conversión se aplique con confianza < 0.75 sin confirmación del usuario.
- **Pedagogy Agent:** revisa que las respuestas del avatar-tutor (sección reutilizada del ERP §17) sean pedagógicamente apropiadas para el grado escolar del alumno — no solo “correctas”, sino explicadas al nivel adecuado.
- **Student-Privacy Agent:** el único con permiso para tocar las políticas de retención/consentimiento de datos de menores (sección 9); cualquier cambio a esa política pasa obligatoriamente por este agente, igual que el Accounting/Audit Agent es el único que toca periodos contables cerrados en el ERP.

11. GATES ESPECÍFICOS +

Gate	Nombre	Evidencia requerida
G31	Confirmación de forma	Ninguna conversión sketch→3D con confianza <0.75 se aplicó sin clic explícito del usuario
G32	Consentimiento de menor	Ninguna función que capture biometría/video de un estudiante está activa sin consentimiento registrado del tutor
G33	Latencia de colaboración	p95 de sincronización de trazo entre usuarios < 150ms en red de salón típica

12. DEPENDENCIAS NUEVAS ESPECÍFICAS

Categoría	Librería	Propósito
CRDT colaborativo	Yjs + y-websocket + y-indexeddb	Documento compartido del lienzo, offline-first
Reconocimiento de escritura	MyScript Interactive Ink SDK (o motor propio entrenado)	Trazo → texto/matemática
Álgebra simbólica	SymPy	Evaluar/graficar expresiones reconocidas
Geometría computacional	SciPy (ConvexHull) + NumPy	Extracción de features del trazo
Render 3D del lienzo	Three.js / React Three Fiber	Igual que el avatar del ERP — se reutiliza la misma dependencia ya elegida
Reutilizados del ERP sin cambio	voice_assistant/*, design-system/avatar/*, platform/concurrency/resource_lock.py, RLS, audit_log, token_vault	Ver secciones citadas — cero reimplementación. La ruta de locking se corrigió: ver AUDITORIA_CONSISTENCIA_ERP_PIZARRA.md, Hallazgo 2 — ya no depende del tooling de Claude Code (agents/runtime/), sino de una librería de plataforma compartida

Resumen: esta pizarra no solo iguala lo que muestra el video de referencia (trazo→forma 3D, escritura matemática, control por voz) — lo supera en tres ejes concretos que el video no demuestra: colaboración real multi-usuario con resolución de conflictos (reutilizando el agente que ya construimos para el ERP), evaluación simbólica real de las expresiones matemáticas (no solo sincronizar el trazo visualmente), y un diseño de privacidad explícito para datos de menores. Y lo hace sin reinventar nada: voz, avatar, agentes, seguridad y design system son los mismos módulos ya contruidos para el ERP — este es un segundo producto sobre la misma plataforma, tal como pediste.

ADENDA V2 — ÁRBOL DE ARCHIVOS COMPLETO, HARDWARE REAL, GRABACIÓN/REPLAY, ACCESIBILIDAD, FLOTA DE DISPOSITIVOS Y MODELO DE DATOS

Se conserva todo lo anterior. Esta adenda cierra los vacíos reales de la V1: no había árbol de archivos, el hardware estaba resuelto en una línea, no existía forma de grabar/repasar una clase, no había accesibilidad, no había gestión de la flota de

pizarras físicas, y no había esquema de datos. Mismo método: análisis de qué debe resistir cada pieza, y luego el archivo/código real.

13. ÁRBOL DE ARCHIVOS COMPLETO DEL PROYECTO

+ V2

```
smart-whiteboard/
├── README.md
├── CLAUDE.md # mismo patrón de contrato maestro que el ERP
├── PRODUCT.md
├── REQUIREMENTS.md
├── ACCEPTANCE.md
├── ARCHITECTURE.md
├── SECURITY.md # hereda de docs/07-security del ERP + sección 9
(menores)
├── .env.example
├── pyproject.toml
├── package.json
├── docker-compose.yml
├── .claude/
│   ├── agents/
│   │   ├── sketch-recognition.md # (definido en §10)
│   │   ├── pedagogy.md # (definido en §10)
│   │   ├── student-privacy.md # (definido en §10)
│   │   ├── device-fleet.md # + V2 (§18)
│   │   └── accessibility.md # + V2 (§17)
│   └── commands/
│       ├── recognition-accuracy-check.md # + V2 – corre la evaluación de §20
│       └── firmware-release.md # + V2
├── backend/app/modules/
│   ├── whiteboard/
│   │   ├── __init__.py
│   │   ├── router.py
│   │   ├── schemas.py
│   │   └── sketch_engine/
│   │       ├── vectorizer.py # (código real, §4)
│   │       └── shape_classifier_model.py # + V2 – wrapper del modelo entrenado (fallback del
│   │           clasificador por reglas)
│   │       └── candidate_ranker.py # + V2 – ordena los 2-3 candidatos mostrados en
│   │           ShapeConfirmationPopover
│   │       ├── math_handwriting/
│   │       │   ├── recognizer.py # (código real, §6)
│   │       │   └── hwr_provider.py
│   │       └── collaboration/
│   │           ├── zone_conflict_bridge.py # (código real, §8)
│   │           ├── session_manager.py # + V2 – quién está conectado a qué room
│   │           └── crdt_document_store.py # + V2 – persistencia del doc Yjs en Postgres
│   └── (snapshots)
│       ├── recording/ # + V2 (§14-15)
│       │   ├── stroke_event_log.py
│       │   ├── audio_sync_writer.py
│       │   └── replay_service.py
│       ├── accessibility/ # + V2 (§16-17)
│       │   ├── live_captions.py
│       │   └── screen_reader_bridge.py
│       ├── device_fleet/ # + V2 (§18-19)
│       │   ├── device_identity.py
│       │   ├── ota_update_manager.py
│       │   └── telemetry_ingest.py
│       ├── academic_integration/ # + V2 (§21)
│       │   ├── gradebook_sync.py
│       │   └── lesson_to_sis_mapper.py
│       └── migrations/versions/
│           ├── 0001_create_lessons_and_strokes.py # (§22)
│           └── 0002_create_device_fleet.py # (§19)
├── frontend/src/design-system/whiteboard/
│   ├── ShapeConfirmationPopover.tsx # (código real, §4)
│   └── useCollaborativeCanvas.ts # (código real, §8)
```

	CanvasRenderer.tsx	# + V2 (\$14) – motor de render con presupuesto de
latencia		
	StrokePredictor.ts	# + V2 (\$14) – suavizado predictivo del trazo
	ReplayTimelineScrubber.tsx	# + V2 (\$15) – reutiliza VideoTimelineCarousel del ERP
	LiveCaptionsOverlay.tsx	# + V2 (\$16)
	DeviceHealthPanel.tsx	# + V2 (\$18)
	firmware/	# + V2 – capa embebida del panel físico
	touch_driver/	
	pen_input_driver/	
	mic_array_beamforming/	
	edge_inference/	# clasificador ligero on-device (\$3, fallback sin red)
	ota_client/	
	hardware/	# + V2 – documentación de ingeniería física (\$13-bis
más abajo)		
	panel-spec.md	
	pen-spec.md	
	mic-array-spec.md	
	thermal-and-power.md	
	tests/	
	recognition_accuracy/	# + V2 (\$20) – dataset de evaluación con ground truth
	collaboration_load/	# + V2 – simula 30 escritores simultáneos
	accessibility/	# + V2 – verifica captions/lectores de pantalla

14. ANÁLISIS PREVIO — HARDWARE FÍSICO (LO QUE ESTABA RESUELTO EN UNA LÍNEA) + v2

1. **Latencia de trazo percibida:** si hay más de ~25ms entre mover el lápiz y ver la tinta, el usuario lo percibe como “lag” — es la queja #1 de cualquier pizarra táctil mala.
2. **Falsos toques por la palma de la mano** (“palm rejection”): un niño apoya la mano al escribir — el panel debe distinguir lápiz activo de contacto accidental.
3. **Captura de voz a distancia:** el profesor camina por el salón; un micrófono simple no capta bien a 4-5 metros con eco de aula → se necesita array de micrófonos con beamforming, no un micrófono de gama de laptop.
4. **Vida útil en uso escolar intensivo:** 6-8 horas/día, uso rudo (niños) → especificación de ciclos táctiles y resistencia al impacto, no solo “pantalla táctil genérica”.

15. ESPECIFICACIÓN DE HARDWARE + v2

hardware/panel-spec.md – resumen de la especificación:

Tecnología táctil:	Infrarrojo (IR) de doble cámara + capa capacitiva proyectada para gesto multitouch – el IR da precisión de lápiz de bajo retardo, la capa capacitiva resuelve gestos con dedo/palma.
Palm rejection:	Discriminación por área de contacto + firma de presión del lápiz activo (el lápiz se identifica por señal propia, no solo por geometría de contacto).
Latencia de trazo:	Presupuesto total < 25ms (input → render), repartido: - Sensor a controlador: ≤ 4ms - Controlador a SO: ≤ 6ms - Predicción/suavizado (\$16): ≤ 5ms - Render en GPU: ≤ 10ms
Resolución de reporte:	≥ 240 puntos/segundo del lápiz activo (no solo la tasa de refresco de pantalla – el trazo se muestrea más rápido de lo que se pinta).
Panel:	LED, ≥ 4K, brillo ajustable automático por luz ambiente (evita deslumbramiento en aulas con ventanas – la referencia no menciona esto).
Ciclos táctiles:	≥ 50 millones de toques por punto (spec de uso escolar intensivo, no de oficina).

hardware/mic-array-spec.md:

Array de micrófonos:	6-8 micrófonos MEMS en arreglo circular + beamforming por software (mismo principio que un smart speaker) – enfoca la captura hacia quien habla, atenúa eco/ruido de fondo del aula.
----------------------	--

Alcance efectivo:	5 metros con SNR utilizable para STT (reutiliza el mismo motor Whisper del `voice_assistant` del ERP, sección 15 del documento ERP).
Cancelación de eco:	AEC (Acoustic Echo Cancellation) porque el propio panel puede reproducir audio (TTS del avatar-tutor) mientras escucha – sin esto, el sistema se "escucharía a sí mismo".
hardware/thermal-and-power.md:	
Cómputo embebido:	SoC con NPU dedicada para el clasificador on-device (§3 del documento base) – el reconocimiento básico de forma NO debe depender de la red para funcionar sin internet.
Disipación:	Diseño fanless o de ventilador de bajo ruido acústico (< 25dB) – un aula no tolera el ruido de un ventilador de servidor.

16. ANÁLISIS PREVIO — RENDERIZADO Y PREDICCIÓN DE TRAZO (SOFTWARE) + V2

1. El presupuesto de 25ms de §15 solo se cumple si el software no agrega su propio retraso — un <canvas> ingenuo repintando todo el lienzo en cada frame no escala con una clase larga (miles de trazos acumulados).
2. La percepción de “instantáneo” se logra en gran parte con **predicción de trazo** (dibujar unos ms hacia adelante de donde probablemente va el lápiz, y corregir cuando llega el dato real) — técnica estándar en apps de dibujo profesional, ausente en la referencia.

17. CÓDIGO REAL — CanvasRenderer.tsx Y StrokePredictor.ts + V2

```
// frontend/src/design-system/whiteboard/StrokePredictor.ts
/**
 * Predicción lineal simple sobre los últimos N puntos para reducir
 * la latencia PERCIBIDA mientras se espera el siguiente evento real
 * del lápiz. Se descarta/corriga en cuanto llega el punto real.
 */
export interface StrokeSample { x: number; y: number; t: number }

export function predictNextPoint(recent: StrokeSample[], lookaheadMs = 12): StrokeSample |
  if (recent.length < 2) return null;
  const [p1, p2] = recent.slice(-2);
  const dt = p2.t - p1.t || 1;
  const vx = (p2.x - p1.x) / dt;
  const vy = (p2.y - p1.y) / dt;
  return { x: p2.x + vx * lookaheadMs, y: p2.y + vy * lookaheadMs, t: p2.t + lookaheadMs }
}

// frontend/src/design-system/whiteboard/CanvasRenderer.tsx
/**
 * Renderiza por capas para no repintar todo el lienzo en cada trazo:
 * - Capa "congelada" (bitmap ya finalizado, se pinta una sola vez)
 * - Capa "en vivo" (solo el trazo activo, se repinta cada frame)
 * Resuelve el punto 1 del análisis §16: una clase con miles de
 * trazos no degrada el frame rate porque la capa congelada no se
 * vuelve a tocar.
 */
import { useRef, useEffect } from "react";

export function CanvasRenderer({ frozenBitmap, liveStroke }: { frozenBitmap: ImageBitmap |
liveStroke: { x: number; y: number }[] }) {
  const canvasRef = useRef<HTMLCanvasElement>(null);

  useEffect(() => {
    const ctx = canvasRef.current?.getContext("2d");
    if (!ctx) return;
    let raf: number;
    const draw = () => {
```

```

        ctx.clearRect(0, 0, ctx.canvas.width, ctx.canvas.height);
        if (frozenBitmap) ctx.drawImage(frozenBitmap, 0, 0); // capa congelada – un solo
drawImage, barato
        ctx.beginPath();
        liveStroke.forEach((p, i) => (i === 0 ? ctx.moveTo(p.x, p.y) : ctx.lineTo(p.x, p.y))
        ctx.stroke(); // solo el trazo activo se re
        raf = requestAnimationFrame(draw);
    };
    raf = requestAnimationFrame(draw);
    return () => cancelAnimationFrame(raf);
}, [frozenBitmap, liveStroke]);

return <canvas ref={canvasRef} className="w-full h-full touch-none" />;
}

```

18. ANÁLISIS PREVIO — GRABACIÓN Y REPASO DE CLASE + V2

1. La referencia no muestra ninguna forma de “volver a ver” la clase — para educación esto es una función central (un alumno que faltó, o que quiere repasar antes de un examen).
2. No se puede grabar solo video de pantalla (pesado, no editable) — hay que grabar el **evento**, no el píxel: cada trazo con su timestamp, cada palabra transcrita del profesor, cada conversión sketch→3D — así el repaso es ligero y se puede saltar/buscar (“llévame al minuto donde explicé la derivada”).
3. Debe sincronizar exactamente audio (voz del profesor) con los trazos — un desfase de más de ~150ms entre lo que se oye y lo que se ve se percibe como mal doblaje.

19. CÓDIGO REAL — stroke_event_log.py Y MODELO DE REPASO + V2

```

# backend/app/modules/whiteboard/recording/stroke_event_log.py
"""
Grabación basada en EVENTOS (no en video), en el mismo espíritu que
el audit_log inmutable del ERP: append-only, cada evento con
timestamp relativo al inicio de la clase, para poder reconstruir
y repasar sin guardar un solo fotograma de video.
"""
from dataclasses import dataclass
from typing import Literal, Any

EventType = Literal["stroke_point", "shape_converted", "math_recognized",
"audio_transcript_chunk"]

@dataclass(frozen=True)
class LessonEvent:
    lesson_id: str
    event_type: EventType
    t_offset_ms: int # tiempo relativo al inicio de la clase, no timestamp absolu
    payload: dict[str, Any]
    author_user_id: str

    async def append_lesson_event(session, event: LessonEvent) -> None:
        # Igual patrón que record_audit_event del ERP: único punto de
        # escritura, en la misma transacción que el cambio que representa.
        await session.execute(
            "INSERT INTO lesson_events (lesson_id, event_type, t_offset_ms, payload, author_us
            "VALUES (:lesson_id, :event_type, :t_offset_ms, :payload, :author_user_id)",
            {"lesson_id": event.lesson_id, "event_type": event.event_type,
            "t_offset_ms": event.t_offset_ms, "payload": event.payload,
            "author_user_id": event.author_user_id},
        )

# backend/app/modules/whiteboard/recording/replay_service.py
async def build_replay_timeline(session, *, lesson_id: str) -> list[dict]:
    """

```

Reconstruye la línea de tiempo completa para el ReplayTimelineScrubber.tsx (reutiliza VideoTimelineCarousel del ERP, sección 9 del documento ERP) — el alumno puede saltar directo a 'shape_converted' o buscar por transcripción de audio.

```
rows = await session.execute(
  "SELECT event_type, t_offset_ms, payload FROM lesson_events "
  "WHERE lesson_id = :lesson_id ORDER BY t_offset_ms ASC",
  {"lesson_id": lesson_id},
)
return [dict(r) for r in rows]
```

Punto 3 del análisis (sincronía audio-trazo) se resuelve por diseño: tanto los `stroke_point` como los `audio_transcript_chunk` comparten el mismo reloj `t_offset_ms` desde el inicio de la clase — no hay dos relojes que puedan desviarse entre sí.

20. ACCESIBILIDAD — CAPTIONS EN VIVO Y LECTOR DE PANTALLA + v2

Ausente por completo en la referencia. Se resuelve reutilizando el mismo motor STT del voice assistant del ERP (sección 15 del documento ERP) para generar subtítulos en vivo de lo que dice el profesor — útil para estudiantes sordos/hipoacúsicos y para los que no tienen el español como lengua materna:

```
// frontend/src/design-system/whiteboard/LiveCaptionsOverlay.tsx
import { useEffect, useState } from "react";

export function LiveCaptionsOverlay({ transcriptStream }: { transcriptStream:
AsyncIterable<string> }) {
  const [currentLine, setCurrentLine] = useState("");

  useEffect(() => {
    (async () => {
      for await (const chunk of transcriptStream) {
        setCurrentLine((prev) => (prev + " " + chunk).slice(-140)); // últimas ~2 líneas v
      }
    })();
  }, [transcriptStream]);

  return (
    <div role="region" aria-live="polite" className="fixed bottom-4 inset-x-0 mx-auto max-w-
text-center bg-black/70 text-white rounded-xl px-4 py-2">
      {currentLine}
    </div>
  );
}
```

El contenido reconocido (figuras 3D, expresiones matemáticas) también expone una descripción textual equivalente (`aria-describedby`) para que un lector de pantalla pueda anunciar “cubo, 8 vértices, convertido desde boceto” en vez de solo silencio sobre un `<canvas>`.

21. ANÁLISIS PREVIO — FLOTA DE PIZARRAS FÍSICAS (MÚLTIPLES SALONES/COLEGIOS) + v2

1. Un colegio puede tener 40 paneles físicos; un distrito, miles → no se puede actualizar firmware manualmente panel por panel.
2. Un panel con firmware corrupto a mitad de actualización no debe quedar “brickeado” en medio de una clase.
3. Mismo problema de identidad de dispositivo que ya resolvimos para drones/flotas en el ERP (sección 19 y 24 del documento ERP) — cada panel se autentica con su propio certificado, nunca credenciales compartidas.

22. CÓDIGO REAL — GESTIÓN DE FLOTA (REUTILIZA EL PATRÓN IoT DEL ERP) + v2

```

# backend/app/modules/whiteboard/device_fleet/ota_update_manager.py
"""
Actualización de firmware por lotes con verificación – mismo
principio de 'reanudable, sin downtime' que rotate_master_key()
del ERP (sección 43 del documento ERP), aplicado aquí a firmware.
"""

from dataclasses import dataclass

@dataclass
class OtaRolloutPlan:
    firmware_version: str
    canary_percentage: int = 5    # primero se despliega a un 5% de paneles, no a todos a

async def rollout_firmware(session, plan: OtaRolloutPlan) -> None:
    canary_devices = await _select_canary_devices(session, plan.canary_percentage)
    for device in canary_devices:
        await _push_update(device, plan.firmware_version)

    # Punto 2 del análisis: espera confirmación de arranque exitoso
    # de cada dispositivo canario antes de expandir al resto – si el
    # canario falla, el rollout se detiene automáticamente.
    if await _canary_healthy(session, canary_devices):
        remaining = await _select_remaining_devices(session, exclude=canary_devices)
        for device in remaining:
            await _push_update(device, plan.firmware_version)
    else:
        await _mark_rollout_halted(session, plan.firmware_version)

```

device_identity.py de este módulo reutiliza literalmente el mismo diseño de field_operations/device_identity.py del ERP (sección 19/24 del documento ERP): certificado por dispositivo, rotación gestionada, cero credenciales compartidas.

23. MODELO DE DATOS COMPLETO – LECCIONES, TRAZOS Y ESTUDIANTES + V2

```

-- backend/app/migrations/versions/0001_create_lessons_and_strokes.py (Alembic)
CREATE TABLE classrooms (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    school_id   UUID NOT NULL,          -- tenant en el sentido del ERP (RLS aplica aq
igual)
    name        TEXT NOT NULL,
    grade_level TEXT NOT NULL
);

CREATE TABLE lessons (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    classroom_id UUID NOT NULL REFERENCES classrooms(id),
    teacher_id  UUID NOT NULL,
    started_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    ended_at    TIMESTAMPTZ,
    retention_until TIMESTAMPTZ NOT NULL    -- política de retención (§9) – obligatoria,
opcional
);

-- Append-only, igual filosofía que audit_log del ERP (sección 29 del documento ERP)
CREATE TABLE lesson_events (
    id          BIGSERIAL PRIMARY KEY,
    lesson_id   UUID NOT NULL REFERENCES lessons(id),
    event_type  TEXT NOT NULL CHECK (event_type IN
('stroke_point', 'shape_converted', 'math_recognized', 'audio_transcript_chunk')),
    t_offset_ms INT NOT NULL,
    payload     JSONB NOT NULL,
    author_user_id UUID NOT NULL
);
CREATE INDEX idx_lesson_events_timeline ON lesson_events (lesson_id, t_offset_ms);
REVOKE UPDATE, DELETE ON lesson_events FROM app_user; -- mismo trigger de inmutabilidad
audit_log

CREATE TABLE student_guardians (
-- soporta el rol "guardian" de so
lectura (§9)

```

```

student_id    UUID NOT NULL,
guardian_id   UUID NOT NULL,
PRIMARY KEY (student_id, guardian_id)
);

```

24. ANÁLISIS PREVIO — EVALUACIÓN DE PRECISIÓN DEL RECONOCIMIENTO + V2

Sin un umbral medible, “el reconocimiento funciona bien” es una afirmación sin sustento — igual que en el ERP no se acepta “cobertura alta” sin definir el umbral por proyecto (sección 9 del documento ERP), aquí se exige lo mismo para precisión de reconocimiento.

25. GATES ADICIONALES V2 + V2 (se suman a G31-G33, ninguno se elimina)

Gate	Nombre	Evidencia requerida
G34	Precisión de reconocimiento	Matriz de confusión sobre el dataset de tests/recognition_accuracy/: recall $\geq$ 90% para las formas soportadas, con umbral de confianza documentado (no “funciona bien” sin número)
G35	Presupuesto de latencia de trazo	p95 input→render < 25ms medido en hardware real (§15), no solo en simulador
G36	Rollout de firmware seguro	Ningún despliegue de firmware a flota completa sin fase canario exitosa (§22)
G37	Accesibilidad de contenido generado	Toda figura/expresión reconocida tiene descripción textual equivalente para lector de pantalla

Resumen V2: se agregó el árbol de archivos completo (antes inexistente), la especificación real de hardware (panel IR+capacitivo, array de micrófonos con beamforming, presupuesto de latencia de 25ms desglosado por etapa — antes resuelto en una sola línea genérica), el motor de renderizado por capas con predicción de trazo, grabación/repaso de clase basada en eventos (no video) con el mismo patrón append-only del audit_log del ERP, accesibilidad real (captions en vivo reutilizando el STT del ERP + descripciones para lector de pantalla), gestión de flota de paneles físicos con rollout canario de firmware (mismo principio que la rotación de claves del ERP), el esquema SQL completo de lecciones/trazos/tutores, y 4 gates nuevos con umbrales medibles en vez de promesas. Ningún vacío señalado queda sin mecanismo concreto.

ADENDA V3 — RESILIENCIA DE PUNTA A PUNTA: QUÉ PASA CUANDO ALGO FALLA, ESCALAMIENTO, OBSERVABILIDAD Y LOS DOS ARCHIVOS QUE QUEDARON SOLO REFERENCIADOS

Se conserva todo V1-V2. El enfoque de esta adenda es distinto a las anteriores: no es “qué función nueva agregar”, es “**qué pasa cuando cada pieza ya construida falla**” — porque un sistema que solo funciona en el camino feliz es el que “se cae” en producción real, con 40 paneles y una clase en curso.

26. ANÁLISIS PREVIO — MATRIZ DE MODOS DE FALLA (LA PIEZA QUE FALTABA DE VERDAD) + v3

Si esto falla...	Sin manejo (lo que “se cae”)	Comportamiento exigido
Red del panel se cae a mitad de clase	El profesor pierde su trazo o la app se congela	El panel sigue escribiendo 100% local (offline-first ya definido en §2/§8); al reconectar, el CRDT reconcilia sin pérdida
Servidor de colaboración (Yjs/WebSocket) cae	Todos los alumnos remotos se desconectan de golpe	Reconexión automática con backoff; mientras tanto cada dispositivo sigue en modo “solo local”, nunca bloquea la escritura
Servicio de reconocimiento de forma (sketch engine) no responde	El botón “convertir” se queda cargando indefinidamente, frustra al usuario	Timeout explícito (§28) → el trazo se queda a mano alzada tal cual, nunca se pierde ni se bloquea la clase
Motor de escritura matemática (recognizer.py) falla	Igual que arriba, aplicado a fórmulas	Se guarda el trazo como anotación visual válida sin capacidad de graficar — ya definido en §5-6, aquí se formaliza como parte de la matriz
STT del asistente de voz no responde	El botón de voz “muere” sin explicación	Fallback a Azure TTS/STT (ya definido como fallback en el ERP, sección 28) + aviso visual explícito, nunca un botón que no reacciona
Un panel físico se desconecta de la red del colegio (no solo de internet)	El panel queda inservible aunque el resto del salón funcione	El panel sigue operando 100% en modo local (firmware con clasificador on-device, §3/§15) — la pizarra NUNCA depende de la red para la función básica de escribir
El clasificador de forma entrenado (modelo ONNX) no carga o da timeout	Excepción no controlada, crash del proceso de reconocimiento	Fallback automático al clasificador por reglas ya construido en §4 (classify_shape) — dos motores, nunca un solo punto de falla
Actualización de firmware falla a mitad de rollout	Paneles “brickeados” en varias aulas a la vez	Ya cubierto por el rollout canario (§22), aquí se agrega rollback automático si el canario no reporta salud en N minutos
Pico de uso (examen simultáneo en 40 salones)	Backend se satura, tiempos de respuesta se disparan	Autoscaling por sharding de “sala” (§30) + rate limiting específico, nunca un monolito que se cae entero por un pico local
Costo de inferencia de IA se dispara (miles de paneles llamando al mismo tiempo)	Factura sorpresa o corte de servicio de golpe	Circuit breaker de costo con degradación progresiva (§30), no corte abrupto

Esta tabla es la pieza central de la adenda: cada fila de abajo resuelve una fila de esta matriz con código real, no con una promesa.

27. CÓDIGO REAL — shape_classifier_model.py (EL ARCHIVO QUE QUEDÓ SOLO REFERENCIADO) + v3

SUPERADO POR LA §37 (V4) – VER NOTA

△ **Hallazgo de autorrevisión contra el Apéndice A §8:** la clase ShapeClassifierModel de esta sección (V3) quedó **duplicada** por la versión completa de la sección 37 (V4) — dos definiciones de la misma clase en el mismo documento-contrato, sin marca explícita de cuál usar, es exactamente “concatenating independent implementations”, prohibido por el Apéndice A. La versión de abajo (V3) se conserva únicamente como referencia histórica de por qué se hizo el cambio (mostraba `_run_inference` CON `NotImplementedError` a propósito); **la implementación vigente y la única que debe usarse es la de la sección 37**, que tiene post-procesamiento real, `classify_top_k()` (no `classify()`), y el archivo de labels versionado. Si estás generando código a partir de este documento: ignora el bloque de abajo, usa solo la sección 37.

```
# backend/app/modules/whiteboard/sketch_engine/shape_classifier_model.py
"""
Wrapper del modelo entrenado (ONNX) para clasificación de formas.
NUNCA es el único camino: si falla, se hace fallback automático al
clasificador por reglas de vectorizer.py (classify_shape), que ya
es explicable y no depende de red ni de un modelo cargado.
"""

from __future__ import annotations
import asyncio
import numpy as np
import onnxruntime as ort

from app.modules.whiteboard.sketch_engine.vectorizer import (
    classify_shape, extract_geometric_features, VectorShapeCandidate,
)

MODEL_INFERENCE_TIMEOUT_S = 0.6 # presupuesto de latencia: no puede competir con el budg
25ms                               # de trazo (§15) – esto corre en paralelo, no en el ca
crítico de dibujar

class ShapeClassifierModel:
    def __init__(self, model_path: str):
        self._session: ort.InferenceSession | None = None
        self._model_path = model_path
        self._load_failed = False

    def _lazy_load(self) -> ort.InferenceSession | None:
        if self._session is not None or self._load_failed:
            return self._session
        try:
            self._session = ort.InferenceSession(self._model_path, providers=
["CPUExecutionProvider"])
        except Exception:
            # Punto crítico del análisis §26: un modelo que no carga
            # NUNCA debe tumbar el servicio – se marca y se sigue con
            # el clasificador por reglas para toda la vida del proceso.
            self._load_failed = True
            self._session = None
        return self._session

    async def classify(self, stroke_raster: np.ndarray, points: list) -> VectorShapeCandid
        session = self._lazy_load()
        if session is None:
            return self._fallback(points)

        try:
            result = await asyncio.wait_for(
                asyncio.to_thread(self._run_inference, session, stroke_raster),
                timeout=MODEL_INFERENCE_TIMEOUT_S,
            )
            return result
        except (asyncio.TimeoutError, Exception):
            # Timeout o error de inferencia -> fallback, nunca excepción
            # visible al usuario (fila "clasificador entrenado" de §26).
```

```

        return self._fallback(points)

    def _run_inference(self, session: ort.InferenceSession, raster: np.ndarray) ->
VectorShapeCandidate:
        input_name = session.get_inputs()[0].name
        logits = session.run(None, {input_name: raster.astype(np.float32)[None, ...]}[0])
        # ... mapeo de logits a VectorShapeCandidate con softmax + top-2 candidatos
        # para alimentar el ShapeConfirmationPopover (§4) – se omite el
        # detalle de post-procesamiento por brevedad, la garantía
        # relevante aquí es el timeout+fallback, no la arquitectura de red.
        raise NotImplementedError("post-procesamiento específico del modelo entrenado")

    def _fallback(self, points: list) -> VectorShapeCandidate:
        features = extract_geometric_features(points)
        return classify_shape(features) # el clasificador por reglas de §4, siempre disp

```

28. CÓDIGO REAL — PROTOCOLO DE BEAMFORMING DEL ARRAY DE MICRÓFONOS + v3

Antes solo se mencionaba “beamforming por software” sin algoritmo. Se especifica el método (delay-and-sum, el estándar de la industria para arrays pequeños de bajo costo — no una caja negra):

```

# firmware/mic_array_beamforming/beamformer.py
"""
Delay-and-sum beamforming: la técnica más simple y robusta para un
array circular de 6-8 micrófonos MEMS (hardware §15). No requiere
entrenamiento ni modelo – es matemática de propagación de onda, por
eso corre en el firmware (edge), no en el servidor.
"""

import numpy as np

SPEED_OF_SOUND_M_S = 343.0

def estimate_direction_of_arrival(mic_signals: np.ndarray, mic_positions: np.ndarray,
sample_rate: int) -> float:
    """
    mic_signals: (n_mics, n_samples) – señal cruda de cada micrófono
    mic_positions: (n_mics, 2) – posición x,y de cada micrófono en el array circular
    Retorna el ángulo (radianes) de la fuente de sonido dominante,
    usado para orientar el beam hacia quien está hablando.
    """
    n_mics = mic_signals.shape[0]
    best_angle, best_energy = 0.0, -np.inf

    for angle in np.linspace(0, 2 * np.pi, 72): # resolución de 5°
        steering_vector = np.array([np.cos(angle), np.sin(angle)])
        delays_samples = (mic_positions @ steering_vector) / SPEED_OF_SOUND_M_S * sample_r
        aligned = np.array([
            np.roll(mic_signals[i], int(-delays_samples[i])) for i in range(n_mics)
        ])
        summed = aligned.mean(axis=0)
        energy = np.sum(summed ** 2)
        if energy > best_energy:
            best_energy, best_angle = energy, angle

    return best_angle

def apply_beamforming(mic_signals: np.ndarray, mic_positions: np.ndarray, sample_rate: int
np.ndarray:
    """Enfoca el array hacia la fuente dominante y devuelve la señal
    combinada – esta es la señal que se envía al STT (reutilizado del
    ERP), con mejor SNR que cualquier micrófono individual."""
    angle = estimate_direction_of_arrival(mic_signals, mic_positions, sample_rate)
    steering_vector = np.array([np.cos(angle), np.sin(angle)])
    delays_samples = (mic_positions @ steering_vector) / SPEED_OF_SOUND_M_S * sample_rate
    aligned = np.array([
        np.roll(mic_signals[i], int(-delays_samples[i])) for i in range(mic_signals.shape[
    ])
    ])
    return aligned.mean(axis=0)

```

Nota de honestidad técnica: delay-and-sum es el método correcto para el caso de uso (una fuente dominante — el profesor hablando), no el más sofisticado que existe (MVDR/beamforming adaptativo daría mejor rechazo de ruido pero exige más cómputo del que tiene sentido meter en el firmware de un panel de aula) — es la elección correcta para el presupuesto de hardware, no una limitación oculta.

29. GESTOR DE DEGRADACIÓN CONTROLADA — EL NÚCLEO DE “QUE NO SE CAIGA” + v3

Resuelve de forma centralizada casi toda la matriz de §26: en vez de que cada componente maneje su propia falla de forma aislada (e inconsistente), existe una máquina de estados única que el resto de la UI consulta.

```
// frontend/src/design-system/whiteboard/DegradedModeManager.ts
export type SystemMode =
  | "ONLINE_FULL"           // red, colaboración, IA e inferencia disponibles
  | "DEGRADED_NO_AI"       // hay red/colaboración, pero sketch/math/voice fallaron -> se
en trazo libre
  | "DEGRADED_LOCAL_ONLY"  // sin red -> solo escritura local, sin colaboración ni IA
  | "RECOVERING";          // reconectando, reconciliando CRDT

export class DegradedModeManager {
  private mode: SystemMode = "ONLINE_FULL";
  private listeners: ((mode: SystemMode) => void)[] = [];

  onNetworkLost() {
    this.setMode("DEGRADED_LOCAL_ONLY"); // fila "red del panel" y "servidor de colabora
de §26
  }

  onAiServiceUnavailable() {
    if (this.mode === "ONLINE_FULL") this.setMode("DEGRADED_NO_AI"); // filas de
sketch/math/voice
  }

  onNetworkRestored() {
    this.setMode("RECOVERING");
    // El CRDT (§8) reconcilia en segundo plano; solo se declara
    // ONLINE_FULL cuando la reconciliación confirma éxito, nunca antes.
  }

  onSyncReconciled() {
    this.setMode("ONLINE_FULL");
  }

  private setMode(mode: SystemMode) {
    this.mode = mode;
    this.listeners.forEach((cb) => cb(mode));
  }

  subscribe(cb: (mode: SystemMode) => void) {
    this.listeners.push(cb);
  }

  /** La regla de oro: escribir a mano NUNCA depende del modo. */
  canWriteFreehand(): boolean {
    return true;
  }

  canUseAiFeatures(): boolean {
    return this.mode === "ONLINE_FULL";
  }

  canCollaborate(): boolean {
    return this.mode === "ONLINE_FULL" || this.mode === "RECOVERING";
  }
}

// frontend/src/design-system/whiteboard/DegradedModeBanner.tsx
// Transparencia con el usuario: nunca falla en silencio.
```

```

export function DegradedModeBanner({ mode }: { mode: SystemMode }) {
  if (mode === "ONLINE_FULL") return null;
  const messages: Record<SystemMode, string> = {
    ONLINE_FULL: "",
    DEGRADED_NO_AI: "Reconocimiento de figuras/voz no disponible por ahora – puedes seguir escribiendo con normalidad.",
    DEGRADED_LOCAL_ONLY: "Sin conexión – tu trabajo se guarda localmente y se sincroniza al reconectar.",
    RECOVERING: "Reconectando y sincronizando...",
  };
  return <div role="status" className="banner-warning">{messages[mode]}</div>;
}

```

30. ESCALAMIENTO Y CONTROL DE COSTO DE IA + v3

Resuelve las dos últimas filas de §26 (pico de uso, costo de inferencia disparado):

```

# backend/app/modules/whiteboard/collaboration/session_manager.py – fragmento de sharding
"""
Cada 'room' (salón) es un shard independiente del servidor de
colaboración – un examen simultáneo en 40 salones no compite por
el mismo proceso. Se asigna por hash de classroom_id, igual
filosofía que particionamiento por tiempo ya usado en el ERP
(TIMESCALEDB, sección 12/14 del documento ERP) pero particionado
por espacio (sala) en vez de tiempo.
"""

def assign_shard(classroom_id: str, total_shards: int = 16) -> int:
    return hash(classroom_id) % total_shards

# backend/app/modules/whiteboard/sketch_engine/cost_circuit_breaker.py
"""
Resuelve la última fila de §26: en vez de un corte abrupto cuando el
gasto de inferencia de IA se dispara, degrada progresivamente –
primero baja la frecuencia de auto-sugerencia, después exige
confirmación manual para cada conversión, y solo en el peor caso
apaga el reconocimiento (nunca la escritura, ver §29).
"""

from dataclasses import dataclass
from enum import Enum

class CostTier(Enum):
    NORMAL = "normal"
    THROTTLED = "throttled" # reduce frecuencia de inferencia, no la corta
    MANUAL_ONLY = "manual_only" # requiere gesto explícito, sin sugerencias proactivas
    DISABLED = "disabled" # última instancia – nunca afecta trazo/voz local

@dataclass
class CostBudget:
    daily_limit_usd: float
    spent_today_usd: float

def current_tier(budget: CostBudget) -> CostTier:
    ratio = budget.spent_today_usd / budget.daily_limit_usd
    if ratio < 0.7:
        return CostTier.NORMAL
    if ratio < 0.9:
        return CostTier.THROTTLED
    if ratio < 1.0:
        return CostTier.MANUAL_ONLY
    return CostTier.DISABLED

```

31. OBSERVABILIDAD Y SLOs ESPECÍFICOS + v3

Métrica	SLO	Alerta si...
Latencia de trazo (input→render)	p95 < 25ms (\$15)	p95 > 40ms sostenido 2 min → alerta a Device-Fleet Agent
Disponibilidad del servicio de colaboración	99.9% mensual	Caída de shard detectada → failover automático a otro shard + alerta

Tasa de fallback del clasificador entrenado	Se espera un % base de fallback a reglas; se alerta si sube de forma anómala	Aumento súbito → puede indicar que el modelo ONNX dejó de cargar en producción
Retraso de sincronización CRDT	p95 < 150ms (gate G33, ya definido)	Sostenido > 300ms → revisar sharding/capacidad
Presupuesto de costo de IA	Ver <code>cost_circuit_breaker.py</code>	<code>CostTier != NORMAL</code> → notificación al administrador del distrito, no solo log interno
Paneles fuera de línea	0 esperado durante horario de clase	Panel reporta último heartbeat > 5 min en horario lectivo → alerta al Device-Fleet Agent

Cada panel expone un healthcheck (reutiliza `scripts/healthcheck.py` del patrón ya definido en el ERP) que reporta: versión de firmware, estado de red, últimos 5 minutos de latencia de trazo, y modo actual del `DegradedModeManager`.

32. RECUPERACIÓN ANTE DESASTRES — GRABACIONES DE CLASE + v3

Sección que faltaba del todo: qué pasa si se pierde la base de datos de `lesson_events` (§23).

- **RPO (Recovery Point Objective):** máximo 5 minutos de eventos de clase perdidos — se logra con el mismo mecanismo de backup incremental ya definido en `docs/05-data/backup-restore.md` del ERP, aplicado a la tabla `lesson_events` con snapshot cada 5 minutos vía WAL de PostgreSQL.
- **RTO (Recovery Time Objective):** una clase en curso puede seguir escribiéndose localmente (offline-first, §29) mientras el backend de grabación se restaura — la restauración del historial no bloquea la clase en vivo.
- **Prueba de restauración periódica:** igual criterio que `docs/05-data/disaster-recovery.md` del ERP — un backup que nunca se probó restaurar no cuenta como backup real.

33. GATES FINALES V3 + v3 (se suman a G31-G37, ninguno se elimina)

Gate	Nombre	Evidencia requerida
G38	Degradación sin pérdida	Prueba automatizada: cortar red a mitad de escritura → cero trazos perdidos al reconectar
G39	Fallback de clasificador	Prueba: forzar fallo del modelo ONNX → <code>classify_shape</code> (reglas) responde igual, sin excepción visible
G40	Circuit breaker de costo	Simulación de gasto al 95% del presupuesto diario → sistema pasa a <code>MANUAL_ONLY</code> , nunca corta escritura/voz local
G41	Restauración de desastre probada	Restauración de <code>lesson_events</code> ensayada en los últimos 90 días con RPO/RTO documentados

Resumen V3: se cerró la brecha real señalada — “que no se caiga” — con una matriz explícita de 11 modos de falla y su comportamiento exigido para cada uno, un `DegradedModeManager` central que garantiza que **escribir a mano nunca depende de nada más** (ni red, ni IA, ni servidor), los dos archivos que habían quedado solo referenciados

(shape_classifier_model.py con timeout+fallback automático al clasificador por reglas, y el algoritmo real de beamforming delay-and-sum), sharding de colaboración por salón para picos de uso simultáneo, un circuit breaker de costo de IA con degradación progresiva en vez de corte abrupto, SLOs/alertas concretos, y disaster recovery de las grabaciones de clase con RPO/RTO definidos. Con V1+V2+V3, cada pieza mencionada en el documento tiene explícitamente definido qué pasa cuando falla — no solo qué pasa cuando todo funciona.

ADENDA V4 — POST-PROCESAMIENTO REAL DEL MODELO ONNX (CIERRE DEL NotImplementedError)

Se conserva todo V1-V3. En §27 (V3), `_run_inference` quedó con `raise NotImplementedError(...)` a propósito, marcado como pendiente. Se cierra aquí con el mismo método: primero qué debe resistir, después el código real.

34. ANÁLISIS PREVIO — QUÉ DEBE RESISTIR EL POST-PROCESAMIENTO + v4

1. **Desalineación de labels:** si el modelo se reentrena y el orden de las clases de salida cambia (ej. `cube` pasa de índice 0 a índice 2), un mapeo `if logit_index == 0: return "cube"` hardcodeado en Python se rompe en silencio — el modelo sigue “funcionando” pero clasifica todo mal. El mapeo índice→forma debe viajar **junto al modelo**, no vivir separado en el código.
2. **Preprocesamiento inconsistente:** el modelo se entrenó con un tamaño/normalización de imagen específico; si el raster que le llega en producción no coincide exactamente, la inferencia no trueno pero degrada silenciosamente — hay que validar la forma del tensor de entrada antes de correr la inferencia, no confiar en que “siempre viene bien”.
3. **Confianza mal calibrada:** el softmax de un modelo no siempre refleja la probabilidad real de acierto — puede estar “sobre-confiado”. Sin calibración, el umbral de 0.75 (definido en §3/V1 para no auto-aplicar) puede ser engañoso.
4. **Formas nuevas que el clasificador por reglas no cubre:** el modelo entrenado puede reconocer más formas (esfera, cilindro, prisma, polígono) que el fallback por reglas — el post-procesamiento debe soportar una plantilla de vértices 3D **extensible**, no una serie de `if/elif` por forma.
5. **Top-2 candidatos reales, no inventados:** el `ShapeConfirmationPopover` (§4) necesita 2-3 candidatos con su confianza real — no basta con la clase ganadora, hay que extraer las siguientes más probables del mismo softmax.

35. CÓDIGO REAL — ARCHIVO DE ETIQUETAS VERSIONADO JUNTO AL MODELO + v4

Resuelve el punto 1: el mapeo vive en un artefacto versionado con el modelo, no en código Python separado que puede desincronizarse.

```
// backend/app/modules/whiteboard/sketch_engine/models/shape_classifier_v1_labels.json
{
  "model_version": "shape-clf-v1.3.0",
  "input_shape": [1, 1, 64, 64],
  "normalization": {"mean": 0.5, "std": 0.5},
  "class_index_to_shape": {
    "0": "cube",
    "1": "cone",
    "2": "cylinder",
    "3": "sphere",
    "4": "triangular_prism",
    "5": "polygon"
  }
}
```

36. CÓDIGO REAL — REGISTRO EXTENSIBLE DE PLANTILLAS DE VÉRTICES 3D + v4

Resuelve el punto 4: agregar una forma nueva es registrar una función, no tocar un if/elif creciente.

```
# backend/app/modules/whiteboard/sketch_engine/vertex_templates.py
from __future__ import annotations
import numpy as np
from typing import Callable

VertexTemplateFn = Callable[[], list[tuple[float, float, float]]]

_VERTEX_TEMPLATES: dict[str, VertexTemplateFn] = {}

def register_vertex_template(shape_type: str):
    """Decorador – cada forma nueva se agrega llamando a esta función
    una vez, en cualquier módulo, sin tocar el post-procesamiento."""
    def decorator(fn: VertexTemplateFn) -> VertexTemplateFn:
        _VERTEX_TEMPLATES[shape_type] = fn
        return fn
    return decorator

def get_vertices_for_shape(shape_type: str) -> list[tuple[float, float, float]]:
    template = _VERTEX_TEMPLATES.get(shape_type)
    if template is None:
        return [] # forma reconocida por el modelo pero sin plantilla 3D aún -> se degr
    return template()

@register_vertex_template("cube")
def _cube_vertices() -> list[tuple[float, float, float]]:
    return [(x, y, z) for x in (0, 1) for y in (0, 1) for z in (0, 1)]

@register_vertex_template("cone")
def _cone_vertices() -> list[tuple[float, float, float]]:
    return [(0, 0, 1)] + [(np.cos(a), np.sin(a), 0) for a in np.linspace(0, 2 * np.pi, 16)]

@register_vertex_template("cylinder")
def _cylinder_vertices() -> list[tuple[float, float, float]]:
    top = [(np.cos(a), np.sin(a), 1) for a in np.linspace(0, 2 * np.pi, 16)]
    bottom = [(np.cos(a), np.sin(a), 0) for a in np.linspace(0, 2 * np.pi, 16)]
    return top + bottom

@register_vertex_template("sphere")
def _sphere_vertices(resolution: int = 12) -> list[tuple[float, float, float]]:
    pts = []
    for theta in np.linspace(0, np.pi, resolution):
        for phi in np.linspace(0, 2 * np.pi, resolution):
            pts.append((
                np.sin(theta) * np.cos(phi),
                np.sin(theta) * np.sin(phi),
                np.cos(theta),
            ))
    return pts

@register_vertex_template("triangular_prism")
def _triangular_prism_vertices() -> list[tuple[float, float, float]]:
    base = [(0, 0, 0), (1, 0, 0), (0.5, 0.87, 0)]
    top = [(x, y, 1) for x, y, _ in base]
    return base + top
```

37. CÓDIGO REAL — POST-PROCESAMIENTO

COMPLETO (CIERRA EL NotImplementedError) + V4 ✓ VERSIÓN VIGENTE de shape_classifier_model.py – reemplaza la \$27

```
# backend/app/modules/whiteboard/sketch_engine/shape_classifier_model.py – versión complet
from __future__ import annotations
import asyncio
import json
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import onnxruntime as ort

from app.modules.whiteboard.sketch_engine.vectorizer import (
    classify_shape, extract_geometric_features, VectorShapeCandidate,
)
from app.modules.whiteboard.sketch_engine.vertex_templates import get_vertices_for_shape

MODEL_INFERENCE_TIMEOUT_S = 0.6
MIN_AUTO_APPLY_CONFIDENCE = 0.75      # mismo umbral definido en V1 ($3) – se reutiliza, n
reinventa
TOP_K_CANDIDATES = 3                  # punto 5 del análisis: candidatos reales para el po
de confirmación

@dataclass(frozen=True)
class ModelMetadata:
    model_version: str
    input_shape: tuple[int, int, int, int]
    norm_mean: float
    norm_std: float
    class_index_to_shape: dict[int, str]

def load_model_metadata(labels_json_path: str) -> ModelMetadata:
    """Punto 1 del análisis: el mapeo de clases se lee del artefacto
    versionado junto al modelo – nunca hardcodeado en Python."""
    data = json.loads(Path(labels_json_path).read_text())
    return ModelMetadata(
        model_version=data["model_version"],
        input_shape=tuple(data["input_shape"]),
        norm_mean=data["normalization"]["mean"],
        norm_std=data["normalization"]["std"],
        class_index_to_shape={int(k): v for k, v in data["class_index_to_shape"].items()},
    )

class ShapeClassifierModel:
    def __init__(self, model_path: str, labels_json_path: str):
        self.session: ort.InferenceSession | None = None
        self._model_path = model_path
        self._metadata = load_model_metadata(labels_json_path)      # falla rápido y explícit
JSON no existe
        self._load_failed = False

    def _lazy_load(self) -> ort.InferenceSession | None:
        if self.session is not None or self._load_failed:
            return self.session
        try:
            self.session = ort.InferenceSession(self._model_path, providers=
["CPUExecutionProvider"])
        except Exception:
            self._load_failed = True
            self.session = None
        return self.session

    async def classify_top_k(self, stroke_raster: np.ndarray, points: list) ->
list[VectorShapeCandidate]:
        """Retorna hasta TOP_K_CANDIDATES ordenados por confianza –
        esto es lo que consume ShapeConfirmationPopover.tsx ($4)."""
        session = self._lazy_load()
```

```

        if session is None:
            return [self._fallback(points)]

        try:
            return await asyncio.wait_for(
                asyncio.to_thread(self._run_inference, session, stroke_raster),
                timeout=MODEL_INFERENCE_TIMEOUT_S,
            )
        except (asyncio.TimeoutError, Exception):
            return [self._fallback(points)]

    def _run_inference(self, session: ort.InferenceSession, raster: np.ndarray) ->
list[VectorShapeCandidate]:
        # Punto 2 del análisis: validar la forma del tensor antes de
        # inferir – mejor fallar explícito aquí que degradar en
        # silencio con un raster mal formado.
        expected_shape = self._metadata.input_shape[1:] # sin el batch dim
        if raster.shape != expected_shape:
            raise ValueError(
                f"Raster shape {raster.shape} no coincide con lo esperado por el modelo
{expected_shape} "
                f"(versión {self._metadata.model_version}) – revisar preprocesamiento"
            )

        normalized = (raster.astype(np.float32) - self._metadata.norm_mean) /
self._metadata.norm_std
        input_name = session.get_inputs()[0].name
        logits = session.run(None, {input_name: normalized[None, ...]})[0][0] # (n_classe

        probabilities = self._softmax(logits)
        top_indices = np.argsort(probabilities)[::-1][:TOP_K_CANDIDATES] # punto 5: top-k
no solo el ganador

        candidates = []
        for idx in top_indices:
            shape_type = self._metadata.class_index_to_shape.get(int(idx))
            if shape_type is None:
                continue # índice fuera del mapa conocido -> se ignora, nunca se inventa
nombre de forma

            confidence = float(probabilities[idx])
            vertices = get_vertices_for_shape(shape_type) # punto 4: extensible vía regis
sin if/elif

            candidates.append(VectorShapeCandidate(
                shape_type=shape_type, # type: ignore[arg-type]
                confidence=confidence,
                vertices_3d=vertices,
                editable_handles=self._default_handles_for(shape_type),
            ))

        if not candidates:
            return [self._fallback([])]
        return candidates

    @staticmethod
    def _softmax(logits: np.ndarray) -> np.ndarray:
        exp = np.exp(logits - np.max(logits)) # estabilidad numérica estándar
        return exp / exp.sum()

    @staticmethod
    def _default_handles_for(shape_type: str) -> list[tuple[float, float]]:
        # Manijas de edición por defecto – escala uniforme para todas
        # las formas nuevas hasta que se defina una específica por tipo.
        return [(0, 0), (1, 1)]

    def _fallback(self, points: list) -> VectorShapeCandidate:
        """Punto 3 del análisis, en conjunto con la fila correspondiente
        de la matriz de fallas (§26, V3): si el modelo no está
        disponible o su confianza es poco fiable, el clasificador por
        reglas de vectorizer.py sigue siendo el camino garantizado."""
        features = extract_geometric_features(points)
        return classify_shape(features)

```

38. CÓDIGO REAL — EVALUACIÓN DE CALIBRACIÓN (CIERRA EL GATE G34) + V4

Resuelve el punto 3: no basta con “el modelo dice 0.92 de confianza”, hay que verificar que esa cifra se corresponde con la tasa de acierto real sobre el dataset de evaluación ya referenciado en tests/recognition_accuracy/ (V2, \$20/\$25).

```
# tests/recognition_accuracy/test_model_calibration.py
"""
Verifica que la confianza reportada por el modelo esté razonablemente
calibrada: de todas las predicciones donde el modelo dijo ~0.9 de
confianza, ¿realmente acierta ~90% de las veces? Esto es lo que
sostiene el umbral MIN_AUTO_APPLY_CONFIDENCE = 0.75 de §37 – sin
esta prueba, ese número sería arbitrario.
"""

import numpy as np
import pytest
from app.modules.whiteboard.sketch_engine.shape_classifier_model import ShapeClassifierModel

CONFIDENCE_BUCKETS = [(0.7, 0.8), (0.8, 0.9), (0.9, 1.0)]
MAX_CALIBRATION_ERROR = 0.15 # tolerancia: la tasa de acierto real no puede
                              # desviarse más de 15 puntos de la confianza reportada

@pytest.mark.asyncio
async def test_confidence_is_calibrated(labeled_shape_dataset, shape_classifier_model:
ShapeClassifierModel):
    predictions = []
    for sample in labeled_shape_dataset:
        candidates = await shape_classifier_model.classify_top_k(sample.raster, sample.poi)
        top = candidates[0]
        predictions.append((top.confidence, top.shape_type == sample.true_label))

    for low, high in CONFIDENCE_BUCKETS:
        bucket = [correct for conf, correct in predictions if low <= conf < high]
        if not bucket:
            continue
        actual_accuracy = sum(bucket) / len(bucket)
        expected_accuracy_midpoint = (low + high) / 2
        assert abs(actual_accuracy - expected_accuracy_midpoint) <= MAX_CALIBRATION_ERROR,
            f"Bucket [{low},{high}]: confianza reportada no calibrada "
            f"(precisión real {actual_accuracy:.2f})"
    )
```

Resumen V4: se cerró el NotImplementedError que había quedado pendiente en _run_inference desde V3. El mapeo de clases del modelo ahora viaja versionado junto al artefacto ONNX (nunca hardcodeado y desincronizable), las plantillas de vértices 3D son un registro extensible (agregar esfera/cilindro/prisma/polígono es registrar una función, no tocar un if/elif), el post-procesamiento valida la forma del tensor de entrada antes de inferir, extrae top-3 candidatos reales del softmax (no solo el ganador) para alimentar el popover de confirmación de V1, y se agregó una prueba de **calibración de confianza** que es lo único que le da sustento real al umbral de 0.75 usado desde la V1 — sin ella, ese número era una suposición razonable, no un hecho verificado.

ADENDA V5 — DATASET DE ENTRENAMIENTO/EVALUACIÓN: RECOLECCIÓN Y ETIQUETADO ÉTICO CON MENORES

Se conserva todo V1-V4. Confirmé primero que el documento del ERP (ERP_ENTERPRISE_AUTONOMO_V2_ROBUSTO.md) no tiene pendientes reales — los NotImplementedError que aparecen ahí son de un patrón de interfaz abstracta

(PaymentGateway, sección 30 del ERP), no código incompleto; ese documento queda tal como está. Esta adenda cierra el pendiente real que quedó abierto en la V4: de dónde sale labeled_shape_dataset (usado en test_confidence_is_calibrated, §38) sin comprometer la privacidad de los niños que generan ese trazo.

39. ANÁLISIS PREVIO — POR QUÉ ESTO NO ES UN DETALLE MENOR + v5

1. **El dataset de entrenamiento es, por definición, trazo de niños reales** — es exactamente el tipo de dato que la sección 9 (V1) dijo que había que proteger. Un dataset de entrenamiento mal recolectado viola las mismas reglas de privacidad que ya definimos para el producto, solo que “por atrás” — en el pipeline de ML en vez de en la app.
2. **Sesgo de accesibilidad:** si el dataset solo tiene trazo de niños diestros, sin temblor, de un rango de edad estrecho, el modelo va a fallar sistemáticamente para zurdos, niños con dificultad motriz fina, o edades fuera del rango — eso convierte una limitación técnica en una barrera de accesibilidad real.
3. **Etiquetado con doble verificación:** una sola persona etiquetando “esto es un cubo” introduce error humano sin forma de detectarlo — hace falta acuerdo entre anotadores (inter-annotator agreement), no una sola opinión.
4. **Derecho al olvido de un menor:** un padre/tutor debe poder pedir que el trazo de su hijo se elimine del corpus de entrenamiento — y el sistema debe poder cumplirlo sin tener que reentrenar buscando manualmente esos ejemplos.

40. CÓDIGO REAL — RECOLECCIÓN CON CONSENTIMIENTO COMO PUERTA DE ENTRADA, NO COMO TRÁMITE + v5

```
# backend/app/modules/whiteboard/sketch_engine/dataset_collection.py
"""
Ningún trazo entra al corpus de entrenamiento sin pasar por esta
función. El consentimiento no es una casilla marcada una vez al
inicio del año escolar – se verifica en cada aportación, porque un
tutor puede revocarlo en cualquier momento (punto 4 del análisis).
"""

from __future__ import annotations
from dataclasses import dataclass
from datetime import datetime, timezone
import hashlib

@dataclass(frozen=True)
class TrainingSampleCandidate:
    student_id: str
    stroke_points: list[dict]
    proposed_label: str
    classroom_id: str

class ConsentNotGrantedError(Exception):
    pass

async def submit_for_training_corpus(session, candidate: TrainingSampleCandidate) -> str |
consent = await _get_active_consent(session, candidate.student_id)
if consent is None or not consent.allows_training_data_use:
    # Punto 1 del análisis: sin consentimiento activo, el trazo
    # se usa para la clase (reconocimiento en vivo) pero JAMÁS
    # se copia al corpus de entrenamiento – dos usos distintos
    # del mismo dato, con puertas de permiso independientes.
    return None

anonymized_id = _pseudonymize(candidate.student_id, candidate.classroom_id)
sample_id = await _store_anonymized_sample(
    session,
    pseudonym=anonymized_id,          # nunca se guarda el student_id real junto al tr
    entrenamiento
```

```

        stroke_points=candidate.stroke_points,
        proposed_label=candidate.proposed_label,
        collected_at=datetime.now(timezone.utc),
    )
    return sample_id

def _pseudonymize(student_id: str, classroom_id: str) -> str:
    # Hash unidireccional – permite borrar TODOS los ejemplos de un
    # estudiante si retira el consentimiento (se puede recalcular el
    # hash a partir del student_id real para ubicarlos), pero el
    # archivo de entrenamiento en sí nunca contiene el ID real.
    salt = classroom_id.encode()
    return hashlib.sha256(student_id.encode() + salt).hexdigest()

async def revoke_consent_and_purge(session, *, student_id: str, classroom_id: str) -> int:
    """Punto 4: derecho al olvido – borra del corpus todo lo asociado
    al pseudónimo derivado de este estudiante, y retorna cuántas
    muestras se eliminaron (evidencia auditable de cumplimiento)."""
    pseudonym = _pseudonymize(student_id, classroom_id)
    deleted_count = await _delete_samples_by_pseudonym(session, pseudonym)
    await _mark_consent_revoked(session, student_id)
    return deleted_count

```

41. CÓDIGO REAL — AUGMENTACIÓN SINTÉTICA (REDUCE LA NECESIDAD DE RECOLECTAR TRAZO REAL DE NIÑOS) + v5

La forma más directa de resolver el punto 1 del análisis es necesitar **menos** datos reales de menores, no solo protegerlos mejor. Se genera variación sintética a partir de un núcleo pequeño de ejemplos ya consentidos:

```

# backend/app/modules/whiteboard/sketch_engine/synthetic_augmentation.py
"""
Genera variaciones sintéticas (jitter geométrico, rotación leve,
simulación de temblor motriz) a partir de un ejemplo real consentido,
para que el dataset de entrenamiento dependa menos de recolectar
miles de trazos reales de niños distintos. También ataca
directamente el punto 2 del análisis (sesgo de accesibilidad):
se simula temblor/imprecisión motriz a propósito, para que el
modelo no falle sistemáticamente con niños que tienen dificultad
de motricidad fina.
"""

import numpy as np

def augment_stroke(points: np.ndarray, *, n_variants: int = 5, simulate_motor_tremor: bool
False) -> list[np.ndarray]:
    variants = []
    for _ in range(n_variants):
        variant = points.copy()
        variant += np.random.normal(0, 1.5, variant.shape) # jitter geométrico le
        angle = np.random.uniform(-8, 8) * np.pi / 180
        rotation = np.array([np.cos(angle), -np.sin(angle)], [np.sin(angle), np.cos(angle)
        variant[:, :2] = variant[:, :2] @ rotation.T

        if simulate_motor_tremor:
            # Punto 2 del análisis: el dataset DEBE incluir ejemplos
            # con temblor simulado a propósito, no solo trazo "ideal".
            tremor = np.cumsum(np.random.normal(0, 0.8, variant.shape), axis=0)
            variant += tremor * 0.3

        variants.append(variant)
    return variants

```

42. CÓDIGO REAL — ACUERDO ENTRE ANOTADORES (CIERRA EL PUNTO 3) + v5

```

# backend/app/modules/whiteboard/sketch_engine/labeling_quality.py
"""
Ningún ejemplo entra al set de entrenamiento "de verdad" (más allá
de estar en cuarentena) sin que al menos 2 anotadores independientes
coincidan en la etiqueta. Mide el acuerdo con Cohen's Kappa, el
estándar para este tipo de verificación – no un simple "¿coinciden
o no?" sin corrección por azar.
"""

from collections import Counter

def cohens_kappa(labels_annotator_a: list[str], labels_annotator_b: list[str]) -> float:
    assert len(labels_annotator_a) == len(labels_annotator_b)
    n = len(labels_annotator_a)
    observed_agreement = sum(a == b for a, b in zip(labels_annotator_a, labels_annotator_b))

    all_labels = set(labels_annotator_a) | set(labels_annotator_b)
    count_a = Counter(labels_annotator_a)
    count_b = Counter(labels_annotator_b)
    expected_agreement = sum((count_a[l] / n) * (count_b[l] / n) for l in all_labels)

    if expected_agreement == 1.0:
        return 1.0
    return (observed_agreement - expected_agreement) / (1 - expected_agreement)

MIN_ACCEPTABLE_KAPPA = 0.75 # umbral estándar de la literatura para "acuerdo sustancial"

def sample_is_trustworthy(kappa_for_batch: float) -> bool:
    return kappa_for_batch >= MIN_ACCEPTABLE_KAPPA

```

43. GATE FINAL + V5

Gate	Nombre	Evidencia requerida
G42	Corpus de entrenamiento consentido	Cero muestras en el dataset de entrenamiento sin <code>consent.allows_training_data_use = true</code> verificado al momento de la recolección
G43	Calidad de etiquetado	Cohen's Kappa ≥ 0.75 entre anotadores para cada lote antes de incorporarlo al set de entrenamiento
G44	Derecho al olvido cumplible	<code>revoke_consent_and_purge()</code> probado end-to-end: retorna el conteo real de muestras eliminadas, no un éxito genérico

Resumen V5: se cerró el último pendiente real señalado (de dónde sale el dataset de evaluación de la V4) sin convertirlo en una fuente oculta de riesgo de privacidad. El consentimiento se verifica en cada aportación (no una vez al año), el trazo de entrenamiento se pseudonimiza con hash unidireccional (permite borrado dirigido sin guardar identidad real), la augmentación sintética reduce la dependencia de recolectar miles de trazos reales de niños y además ataca directamente el sesgo de accesibilidad simulando temblor motriz a propósito, y ningún lote se incorpora sin acuerdo verificado entre anotadores (Cohen's Kappa ≥ 0.75). Con V1 a V5, el pipeline completo de este producto —desde el trazo en el panel físico hasta el dato que entrena el modelo que reconoce ese trazo— tiene su privacidad y su resiliencia cerradas de punta a punta, sin cabos sueltos declarados como “para después”.

ADENDA V6 — BACKLOG PROPIO Y ORQUESTACIÓN AUTOMÁTICA (LA

PIZARRA NUNCA TUVO UNO)

44. ANÁLISIS PREVIO — EL DETALLE QUE SE HABÍA QUEDADO FUERA + V6

El mecanismo de backlog automático + loop autónomo continuo (module_backlog.yaml, scheduler.py, loop_controller.py) se construyó únicamente en el ERP (V12-V13 de ese documento). La Pizarra heredó el **Apéndice A** (reglas de arquitectura) pero nunca el **mecanismo de construcción automática** — por eso solo tenía 2 prompts (sketch, dataset) mientras que el documento ya contenía código real de al menos 6 piezas más sin ningún backlog ni prompt: zone_conflict_bridge.py, stroke_event_log.py/replay_service.py, ota_update_manager.py, beamformer.py, y el sharding de session_manager.py. Se corrige aquí, reutilizando literalmente el mismo diseño del ERP (mismo formato de module_backlog.yaml, mismo scheduler.py) — no se reinventa un mecanismo paralelo.

45. agents/runtime/tasks/module_backlog.yaml PROPIO DE LA PIZARRA + V6

```
# smart-whiteboard/agents/runtime/tasks/module_backlog.yaml + NUEVO
# Mismo formato y mismo scheduler.py que erp-enterprise/ (reutilización
# de plataforma real, no un mecanismo paralelo) — se importa el
# scheduler como dependencia compartida, ver whiteboard_platform/.
modules:
  - id: sketch_engine
    depends_on: []
    gates: [G39]
    scope: "vectorizer.py + vertex_templates.py + shape_classifier_model.py — ya con código
completo en §4/§27/§37"
    must_not: ["implementar el modelo ONNX entrenado real sin dataset (ver dataset_collect

  - id: math_handwriting
    depends_on: [sketch_engine]
    gates: []
    scope: "recognizer.py — código completo en §6"
    must_not: ["bloquear el trazo si el parseo LaTeX falla — debe guardarse como anotación
igual"]

  - id: collaboration
    depends_on: [sketch_engine]
    gates: [G33]
    scope: "useCollaborativeCanvas.ts (CRDT/Yjs) + zone_conflict_bridge.py + session_manag
(sharding por salón, §30) — código completo en §8, IMPORTANTE: zone_conflict_bridge.py usa
platform/concurrency/resource_lock.py del ERP (ver AUDITORIA_CONSISTENCIA_ERP_PIZARRA.md), nunca
el tooling agents/runtime/ de ningún repo"

  - id: audio_beamforming
    depends_on: []
    gates: []
    scope: "firmware/mic_array_beamforming/beamformer.py (§28, delay-and-sum) — corre en
firmware/edge, no en el backend; validar contra hardware real del array de micrófonos (§15), no
solo simulación"
    must_not: ["implementar MVDR u otro beamforming adaptativo — la decisión ya está tomad
(delay-and-sum) por presupuesto de cómputo del panel, ver justificación en §28"]
    must_not: ["reimplementar el locking — importar de la librería de plataforma compartid

  - id: recording_replay
    depends_on: [collaboration]
    gates: []
    scope: "stroke_event_log.py + replay_service.py — código completo en §19, mismo patrón
append-only que audit_log del ERP"
    must_not: ["grabar video/píxel en vez de eventos — la grabación es por evento, no por
fotograma"]

  - id: accessibility
    depends_on: [sketch_engine]
    gates: [G37]
    scope: "LiveCaptionsOverlay.tsx — código completo en §20, reutiliza el STT del
```

```

voice_assistant del ERP"
    must_not: ["crear un motor STT propio en vez de reutilizar el del ERP"]

- id: device_fleet
  depends_on: []
  gates: [G36]
  scope: "ota_update_manager.py – código completo en §22, rollout canario obligatorio"
  must_not: ["desplegar firmware a la flota completa sin fase canario exitosa primero"]

- id: resilience_layer
  depends_on: [sketch_engine, collaboration]
  gates: [G38, G40]
  scope: "DegradedModeManager.ts + cost_circuit_breaker.py – código completo en §29/§30"
  must_not: ["dejar que escribir a mano dependa de red o de IA – canWriteFreehand() siem
true"]

- id: dataset_collection
  depends_on: [sketch_engine]
  gates: [G42, G43, G44]
  scope: "dataset_collection.py + synthetic_augmentation.py + labeling_quality.py – código
completo en §40-42"
  must_not: ["entrenar el modelo en esta tarea", "recolectar biometría/video"]

```

46. loop_controller.py DE LA PIZARRA — REUTILIZA EL DEL ERP, NO LO DUPLICA + v6

```

# smart-whiteboard/agents/runtime/orchestrator/loop_controller.py  + NUEVO
"""
NO reimplementa run_autonomous_loop() – lo importa como paquete
compartido de plataforma (mismo principio que resource_lock.py entre
ERP y Pizarra). Solo apunta a SU backlog propio.
"""

from whiteboard_platform.orchestrator_core import run_autonomous_loop # misma lógica que
agents/runtime/orchestrator/loop_controller.py del ERP, extraída como paquete compartido –
corrección adicional: ni el ERP debería tener esa lógica "propia" si dos productos la necesitan
igual

async def run_whiteboard_loop(completed_ids: set[str]) -> None:
    await run_autonomous_loop(
        backlog_path="agents/runtime/tasks/module_backlog.yaml",
        completed_ids=completed_ids,
    )

```

Nota de consistencia con el ERP: esto implica una corrección retroactiva menor al ERP también — `run_autonomous_loop()` (ERP, V12 §56) debería vivir en un paquete de plataforma compartido (`whiteboard_platform.orchestrator_core` o un nombre más neutral como `platform_core.orchestrator`), no duplicado entre los dos repos. Se deja documentado aquí como ajuste pendiente para la próxima vez que se toque el `loop_controller.py` del ERP — mismo criterio de “lo señalo en vez de dejarlo implícito” que el resto de este documento.

47. LOS 6 PROMPTS QUE FALTABAN + v6

Mismo criterio que el ERP: cada entrada del backlog tiene su prompt humano-legible correspondiente. Se listan los que faltaban (formato idéntico a `PROMPT_ARRANQUE_PIZARRA_SKETCH.md`, omitido el cuerpo completo aquí por espacio — mismo patrón: qué implementar de código ya dado, qué NO hacer, qué confirmar al terminar):

- `PROMPT_ARRANQUE_PIZARRA_COLLABORATION.md` — depende de `sketch_engine`; implementa CRDT + `zone_conflict_bridge.py` usando `resource_lock.py` del ERP.
- `PROMPT_ARRANQUE_PIZARRA_RECORDING.md` — depende de `collaboration`; grabación por eventos, nunca por video.
- `PROMPT_ARRANQUE_PIZARRA_ACCESSIBILITY.md` — depende de `sketch_engine`; reutiliza STT del ERP, no crea uno propio.
- `PROMPT_ARRANQUE_PIZARRA_DEVICE_FLEET.md` — sin dependencias; rollout canario obligatorio de firmware.
- `PROMPT_ARRANQUE_PIZARRA_RESILIENCE.md` — depende de `sketch_engine+collaboration`; la regla de oro `canWriteFreehand()` siempre true debe tener prueba explícita.

- `PROMPT_ARRANQUE_PIZARRA_DATASET.md` — **ya existía** desde V5, se mantiene sin cambios.
-

Resumen V6: se encontró y corrigió un detalle real de alcance mayor a los anteriores — la Pizarra tenía código completo para 6+ piezas (colaboración, grabación, accesibilidad, flota de dispositivos, resiliencia) sin ningún mecanismo de backlog ni prompt, a diferencia del ERP que sí lo tenía desde V12. Se agregó `module_backlog.yaml` propio reutilizando el mismo `scheduler.py/loop_controller.py` de plataforma (no un mecanismo paralelo), y se identificó de paso una mejora pendiente para el ERP: su propio `run_autonomous_loop()` debería vivir en un paquete de plataforma compartido en vez de duplicarse si dos productos lo necesitan igual — queda documentado como ajuste futuro, no aplicado a ciegas en esta misma sesión para no exceder el alcance de esta corrección.

ADENDA V7 — SEGUNDA PASADA DE DETALLE: `session_manager.py` Y `beamformer.py` QUEDARON SOLO EN PROSA, NO EN EL BACKLOG REAL

Al escribir el análisis de la V6 (§44), mencioné `beamformer.py` y el sharding de `session_manager.py` como piezas que necesitaban backlog — pero el YAML real que escribí a continuación no los incluyó. Es el mismo tipo de error que ya veníamos corrigiendo en el ERP (declarar algo en prosa sin que el dato real lo refleje). Corregido: `session_manager.py` se agregó al scope de `collaboration` (comparten el mismo archivo de colaboración), y `beamformer.py` obtuvo su propia entrada `audio_beamforming` — no encajaba naturalmente en ninguna de las 8 tareas existentes porque corre en `firmware/edge`, no en el backend ni el frontend, y no depende de ninguna otra pieza del backlog.

Nota honesta de proceso: esto confirma que “revisar mientras se escribe” tiene un límite — encontré este segundo caso solo al inventariar explícitamente cada archivo con código real contra cada entrada de backlog, no releyendo el texto. Es la misma razón por la que el gate G47/V23 (detección automática árbol→código) existe: un script que compara listas es más confiable que una relectura humana, incluso la mía.